EAST POINT COLLEGE OF ENGINEERING & TECHNOLOGY

AN AUTONOMOUS INSTITUTION, AFFILIATED TO VTU, APPROVED BY AICTE
ACCREDITED BY NATIONAL BOARD OF ACCREDITATION (NBA) - CSE, ECE & ISE

NAAC GRADE A

Department of Computer Science & Engineering

Academic Year: 2025-26

LABORATORY MANUAL

Semester : III

Subject : Data Structures & Applications Laboratory

Subject Code : 24CSL35

NAME: _____

USN: _____

SECTION: _____

PROGRAM OUTCOMES

Engineering Graduates will able to

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WKl to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5).

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and +modem engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WKl, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

POl0: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Department of Computer Science & Engineering

INSTITUTE VISION AND MISSION

VISION

The East Point College of Engineering and Technology aspires to be a globally acclaimed institution, recognized for excellence in engineering education, applied research and nurturing students for holistic development.

MISSION

M1: To create engineering graduates through quality education and to nurture innovation, creativity and excellence in teaching, learning and research

M2: To serve the technical, scientific, economic and societal developmental needs of our communities

M3: To induce integrity, teamwork, critical thinking, personality development and ethics in students and to lay the foundation for lifelong learning

EAST POINT COLLEGE OF ENGINEERING & TECHNOLOGY

AN AUTONOMOUS INSTITUTION, AFFILIATED TO VTU, APPROVED BY AICTE
ACCREDITED BY NATIONAL BOARD OF ACCREDITATION (NBA) - CSE, ECE & ISE

NAAC GRADE A

Department of Computer Science & Engineering

DEPARTMENT VISION AND MISSION

VISION

The department aspires to be a Centre of excellence in Computer Science & Engineering to develop competent professionals through holistic development.

MISSION

M1: To create successful Computer Science Engineering graduates through effective pedagogies, the latest tools and technologies, and excellence in teaching and learning.

M2: To augment experiential learning skills to serve technical, scientific, economic, and social developmental needs.

M3: To instil integrity, critical thinking, personality development, and ethics in students for a successful career in Industries, Research, and Entrepreneurship.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO 1: To produce graduates who can perform technical roles to contribute effectively in software industries and R&D Centre

PEO 2: To produce graduates having the ability to adapt and contribute in key domains of computer science and engineering to develop competent solutions.

PEO 3: To produce graduates who can provide socially and ethically responsible solutions while adapting to new trends in the domain to carve a successful career in the industry

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: To conceptualize, model, design, simulate, analyse, develop, test, and validate computing systems and solve technical problems arising in the field of computer science & engineering.

PSO2: To specialize in the sub-areas of computer science & engineering systems such as cloud computing, Robotic Process Automation, cyber security, big data analytics, user interface design, and IOT to meet industry requirements.

PSO3: To build innovative solutions to meet the demands of the industry using appropriate tools and techniques.

COURSE LEARNING OBJECTIVES

This laboratory course enables students to get practical experience in design, develop, implement, analyze and evaluation/testing of

CLO 1. Dynamic memory management

CLO 2. Linear data structures and their applications such as stacks, queues and lists

CLO 3. Non-Linear data structures and their applications such as trees and graphs

COURSE OUTCOMES

At the end of the course the student will be able to:

CO 1. Analyze various linear and non-linear data structures.

CO 2. Demonstrate the working nature of different types of data structures and their applications.

CO 3. Use appropriate searching and sorting algorithms for the given scenario.

CO 4. Apply the appropriate data structure for solving real world problems.

CO 5. Develop solutions to solve real world problems.

DATA STRUCTURES & APPLICATIONS LABORATORY			
Course Code	24CSL35	**CIE Marks**	50
Teaching Hours/Week (L: T: P)	0:0:2	**SEE Marks**	50
Credits	01	**Total Marks**	100
Contact Hours	40	**Exam Hours**	3
Examination type (SEE)	Practical		

Prerequisites:

The students should have knowledge on

- Basic Programming Skills
- Understanding of Algorithms
- Basic Mathematics
- Problem-Solving Skills

Course Objectives:

This laboratory courses enables students to get practical experience in design, develop, implement, analyze and evaluation/testing of

- Dynamic memory management
- Linear data structures and their applications such as stacks, queues and lists
- Non-Linear data structures and their applications such as trees and graphs

Teaching-Learning Process (General Instructions)

Teachers can use following strategies to accelerate the attainment of the various course outcomes.

1. Chalk and Talk with Black Board
2. ICT based Teaching
3. Demonstration based Teaching

Programs List:

1.	Develop a Program in C for the following: a. Declare a calendar as an array of 7 elements (A dynamically Created array) to represent 7 days of a week. Each Element of the array is a structure having three fields. The first field is the name of the Day (A dynamically allocated String), The second field is the date of the Day (A integer), the third field is the description of the activity for a particular day (A dynamically allocated String). b. Write functions create(), read() and display(); to create the calendar, to read the data from the keyboard and to print weeks activity details report on screen.
2.	Develop a Program in C for the following operations on Strings. a. Read a main String (STR), a Pattern String (PAT) and a Replace String (REP) b. Perform Pattern Matching Operation: Find and Replace all occurrences of PAT in STR with REP if PAT exists in STR. Report suitable messages in case PAT does not exist in STR Support the program with functions for each of the above operations. Don't use Built-in functions.
3.	Develop a menu driven Program in C for the following operations on STACK of Integers (Array Implementation of Stack with maximum size MAX) a. Push an Element on to Stack b. Pop an Element from Stack c. Demonstrate how Stack can be used to check Palindrome d. Demonstrate Overflow and Underflow situations on Stack e. Display the status of Stack f. Exit

	Support the program with appropriate functions for each of the above operations
4	Develop a Program in C for converting an Infix Expression to Postfix Expression. Program should support for both parenthesized and free parenthesized expressions with the operators: +, -, *, /, % (Remainder), ^ (Power) and alphanumeric operands.
5.	Develop a Program in C for the following Stack Applications a. Evaluation of Suffix expression with single digit operands and operators: +, -, *, /, %, ^ b. Solving Tower of Hanoi problem with n disks
6.	Develop a menu driven Program in C for the following operations on Circular QUEUE of Characters (Array Implementation of Queue with maximum size MAX) a. Insert an Element on to Circular QUEUE b. Delete an Element from Circular QUEUE c. Demonstrate Overflow and Underflow situations on Circular QUEUE d. Display the status of Circular QUEUE e. Exit Support the program with appropriate functions for each of the above operations
7	Develop a menu driven Program in C for the following operations on Singly Linked List (SLL) of Student Data with the fields: *USN, Name, Programme, Sem, PhNo* a. Create a SLL of N Students Data by using *front insertion.* b. Display the status of SLL and count the number of nodes in it c. Perform Insertion / Deletion at End of SLL d. Perform Insertion / Deletion at Front of SLL (Demonstration of stack)
8	Develop a menu driven Program in C for the following operations on Doubly Linked List (DLL) of Employee Data with the fields: *SSN, Name, Dept, Designation, Sal, PhNo* a. Create a DLL of N Employees Data by using *end insertion.* b. Display the status of DLL and count the number of nodes in it c. Perform Insertion and Deletion at End of DLL d. Perform Insertion and Deletion at Front of DLL e. Demonstrate how this DLL can be used as Double Ended Queue.
9	Develop a Program in C for the following operations on Singly Circular Linked List (SCLL) with header nodes a. Represent and Evaluate a Polynomial $P(x, y) = 6x^2 y^2 - 4y + 3x^3 y + 2xy^5 - 2x$. b. Find the sum of two polynomials POLY1(x, y) and POLY2(x, y) and store the result in POLYSUM(x, y)
10	Develop a menu driven Program in C for the following operations on Binary Search Tree (BST) of Integers. a. Create a BST of N Integers: 6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2 b. Traverse the BST in Inorder, Preorder and Post Order c. Search the BST for a given element (KEY) and report the appropriate message
11	Develop a Program in C for the following operations on Graph(G) of Cities a. Create a Graph of N cities using Adjacency Matrix. b. Print all the nodes reachable from a given starting node in a digraph using DFS/BFS method
12.	Given a File of N employee records with a set K of Keys (4-digit) which uniquely determine the records in file F. Assume that file F is maintained in memory by a Hash Table (HT) of m memory locations with L as the set of memory addresses (2-digit) of locations in HT. Let the keys in K and addresses in L are Integers. Develop a Program in C that uses Hash function H: K →L as H(K)=K mod m (remainder method), and implement hashing technique to map a given key K to the address space L. Resolve the collision (if any) using linear probing.

Course outcomes (Course Skill Set):

At the end of the course, the student will be able to:

CO1: Analyze various linear and non-linear data structures (PO1-4,11, PSO 1-3)

CO2: Demonstrate the working nature of different types of data structures and their applications (PO1-4,11, PSO 1-3)

CO3: Use appropriate searching and sorting algorithms for the give scenario. (PO1-4,11, PSO 1-3)

CO4: Apply the appropriate data structure for solving real world problems (PO1-4,11, PSO 1-3)

CO5: Develop Solutions to real world problems. (PO1-4,11, PSO 1-3)

Assessment Details (both CIE and SEE):

- Experiment distribution
  - For laboratories having only one part: Students are allowed to pick one experiment from the lot with equal opportunity.
  - For laboratories having PART A and PART B: Students are allowed to pick one experiment from PART A and one experiment from PART B, with equal opportunity.
- Change of experiment is allowed only once and marks allotted for procedure to be made zero of the changed part only.
- Marks Distribution *(Need to change in accordance with university regulations)*
  c) For laboratories having only one part – Procedure + Execution + Viva-Voce: 15+70+15 = 100 Marks
  d) For laboratories having PART A and PART B
    i. Part A – Procedure + Execution + Viva = 6 + 28 + 6 = 40 Marks
    ii. Part B – Procedure + Execution + Viva = 9 + 42 + 9 = 60 Marks

Web links and Video Lectures(e-Resources):

- https://ds1-iiith.vlabs.ac.in/exp/stacks-queues/index.html
- https://ds1-iiith.vlabs.ac.in/exp/linked-list/basics/overview.html
- https://ds1-iiith.vlabs.ac.in/List%20of%20experiments.html
- https://ds1-iiith.vlabs.ac.in/exp/tree-traversal/index.html
- https://ds1-iiith.vlabs.ac.in/exp/tree-traversal/depth-first-traversal/dft-practice.html

Index					
Sl No	**Program List**	**CO**	**PO**	**RBT**	**Page No**
1	Develop a Program in C for the following: a. Declare a calendar as an array of 7 elements (A dynamically Created array) to represent 7 days of a week. Each Element of the array is a structure having three fields. The first field is the name of the Day (A dynamically allocated String), The second field is the date of the Day (A integer), the third field is the description of the activity for a particular day (A dynamically allocated String). b. Write functions create(), read() and display(); to create the calendar, to read the data from the keyboard and to print weeks activity details report on screen.	CO1	PO3	L3	14
2	Develop a Program in C for the following operations on Strings. a. Read a main String (STR), a Pattern String (PAT) and a Replace String (REP) b. Perform Pattern Matching Operation: Find and Replace all occurrences of PAT in STR with REP if PAT exists in STR. Report suitable messages in case PAT does not exist in STR Support the program with functions for each of the above operations. Don't use Built-in functions.	CO1	PO3	L3	19
3	Develop a menu driven Program in C for the following operations on STACK of Integers (Array Implementation of Stack with maximum size MAX) a. Push an Element on to Stack b. Pop an Element from Stack c. Demonstrate how Stack can be used to check Palindrome d. Demonstrate Overflow and Underflow situations on Stack e. Display the status of Stack f. Exit Support the program with appropriate functions for each of the above operations	CO1	PO3	L3	23
4	Develop a Program in C for converting an Infix Expression to Postfix Expression. Program should support for both parenthesized and free parenthesized expressions with the operators:+, *,/,%(Remainder), ^(Power) and alphanumeric operands.	CO1	PO3	L3	29

5	Develop a Program in C for the following Stack Applications a. Evaluation of Suffix expression with single digit operands and operators: +, -, *, /, %, ^ b. Solving Tower of Hanoi problem with n disks	CO1	PO3	L3	33
6	Develop a menu driven Program in C for the following operations on Circular QUEUE of Characters (Array Implementation of Queue with maximum size MAX) a. Insert an Element on to Circular QUEUE b. Delete an Element from Circular QUEUE c. Demonstrate Overflow and Underflow situations on Circular QUEUE d. Display the status of Circular QUEUE e. Exit	CO2	PO3	L3	36
7	Develop a menu driven Program in C for the following operations on Singly Linked List (SLL) of Student Data with the fields: USN, Name, Programme, Sem, Ph No a. Create a SLL of N Students Data by using front insertion. b. Display the status of SLL and count the number of nodes in it c. Perform Insertion / Deletion at End of SLL d. Perform Insertion / Deletion at Front of SLL (Demonstration of stack)	CO2	PO3	L3	41
8	Develop a menu driven Program in C for the following operations on Doubly Linked List (DLL) of Employee Data with the fields: SSN, Name, Dept, Designation, Sal, PhNo a. Create a DLL of N Employees Data by using end insertion. Display the status of DLL and count the number of nodes in it b. Perform Insertion and Deletion at End of DLL c. Perform Insertion and Deletion at Front of DLL d. Demonstrate how this DLL can be used as Double Ended Queue.	CO3	PO3	L3	46

9	Develop a Program in C for the following operations on Singly Circular Linked List (SCLL) with header nodes a. Represent and Evaluate a Polynomial $P(x,y) = 6x^2y^2 - 4y + 3x^3y + 2xy^5 - 2xy$ b. Find the sum of two polynomials POLY1(x, y) and POLY2(x, y) and store the result in POLYSUM(x,y) Support the program with appropriate functions for each of the above operations	CO3	PO3	L3	51
10	Develop a menu driven Program in C for the following operations on Binary Search Tree (BST) of Integers. a. Create a BST of N Integers: 6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2 b. Traverse the BST in Inorder, Preorder and Post Order c. Search the BST for a given element (KEY) and report the appropriate message.	CO3	PO3	L3	59
11	Develop a program in C for the following operations on Graph (G) of cities a. Create a Graph of N cities using Adjacency Matrix b. Print all the nodes reachable from a given starting node in a diagraph using DFS/BFS Method	CO4	PO3	L3	66
12	Given a File of N employee records with a set K of Keys (4-digit) which uniquely determine the records in file F. Assume that file F is maintained in memory by a Hash Table (HT) of m memory locations with L as the set of memory addresses (2-digit) of locations in HT. Let the keys in K and addresses in L are Integers. Develop a Program in C that uses Hash function H: K →L as H(K)=K mod m (remainder method), and implement hashing technique to map a given key K to the address space L. Resolve the collision (if any) using linear probing.	CO5	PO3	L3	74

Course Articulation Matrix

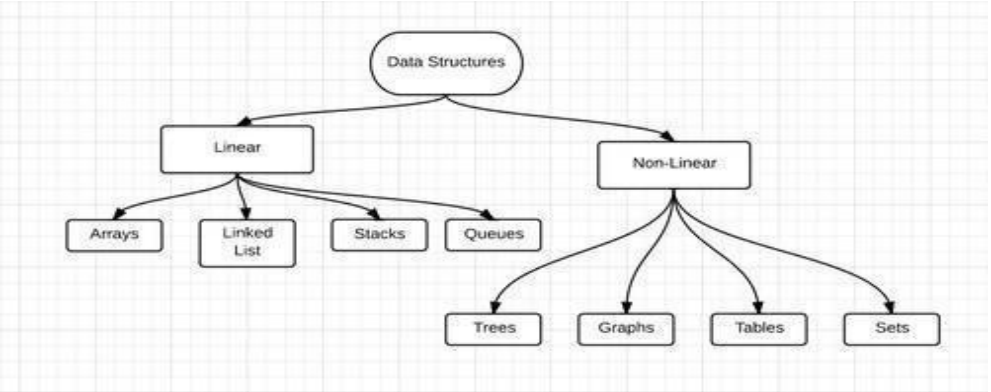
Cos	POs											PSOs		
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	3	3	3	1							2	3	3	3
CO2	3	3	3	2							2	3	3	3
CO3	3	3	3	2							2	3	3	2
CO4	3	3	3	2							2	3	3	3
CO5	3	3	3	2							2	3	3	3

3 - High Correlation 2 - Medium Correlation 1 – Low Correlation

INTRODUCTION TO DATA STRUCTURES

Data Structure is defined as the way in which data is organized in the memory location. There are 2 types of data structures:

Linear Data Structure:



In linear data structure all the data are stored linearly or contiguously in the memory. All the data are saved in continuously memory locations and hence all data elements are saved in one boundary. A linear data structure is one in which we can reach directly only one element from another while travelling sequentially. The main advantage, we find the first element, and then it's easy to find the next data elements. The disadvantage, the sizeof the array must be known before allocating the memory.

The different types of linear data structures are:

- Array
- Stack
- Queue
- Linked List

Non-Linear Data Structure:

Every data item is attached to several other data items in a way that is specific for reflecting relationships. The data items are not arranged in a sequential structure.

The various types of non linear data structures are:

- Trees
- Graphs

EXPERIMENT - 01

Develop a Program in C for the following:

a. Declare a calendar as an array of 7 elements (A dynamically Created array) to represent 7 days of a week. Each Element of the array is a structure having three fields. The first field is the name of the Day (A dynamically allocated String), The second field is the date of the Day (A integer), the third field is the description of the activity for a particular day (A dynamically allocated String).
b. Write functions create(), read() and display(); to create the calendar, to read the data from the keyboard and to print weeks activity details report on screen.

ABOUT THE EXPERIMENT:

An **Array** is a collection of similar / same elements. In this experiment the array can be represented as one / single dimensional elements.

Menu driven program in c - language to perform various array operations are implemented with the help of user defined functions as followings;

a. create()
b. display()
c. exit()

PROGRAM CODE:

```
#include <stdio.h>
#include <stdlib.h>
struct Day
    {
       char *name;
       int date;
       char *activity;
    };
    struct Day create()
    {
       struct Day day;
       day.name = malloc(20 * sizeof(char));
       day.activity = malloc(100 * sizeof(char));
       printf("Enter the day name: ");
       scanf(" %19s", day.name);
       printf("Enter the date: ");
       scanf("%d", &day.date);
       printf("Enter the activity for the day: ");
       scanf(" %99[^\n]", day.activity);
       return day;
    }
    void read(struct Day calendar[], int size)
     {
       for (int i = 0; i < size; i++)
       {
          printf("\n--- Enter details for Day %d ---\n", i + 1);
          calendar[i] = create();
       }

    }
```

```
void display(struct Day calendar[], int size)
 {
   printf("\nWeekly Activity Details:\n");
   for (int i = 0; i < size; i++)
   {
      printf("Day %d: %s\n", i + 1, calendar[i].name);
      printf("Date: %d\n", calendar[i].date);
      printf("Activity: %s\n\n", calendar[i].activity);
   }
}
int main()
 {
   int weekSize = 7;
   struct Day calendar[weekSize];
   read(calendar, weekSize);
    display(calendar, weekSize);
   // Free allocated memory
   for (int i = 0; i < weekSize; i++)
    {
      free(calendar[i].name);
      free(calendar[i].activity);
    }
   return 0;
}
```

Sample Output:

```
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$ gcc p1.c
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$ ./a.out

--- Enter details for Day 1 ---
Enter the day name: Monday Enter
the date: 101025
Enter the activity for the day: Reading

--- Enter details for Day 2 ---
Enter the day name: Tuesday Enter
the date: 111025
Enter the activity for the day: Coding

--- Enter details for Day 3 ---
Enter the day name: Wednesday
Enter the date: 121025
Enter the activity for the day: Painting

--- Enter details for Day 4 ---
```

Enter the day name: Thursday
Enter the date: 131025
Enter the activity for the day: Watching Movie
--- Enter details for Day 5 --- Enter
the day name: Friday Enter the
date: 141025
Enter the activity for the day: Reading
--- Enter details for Day 6 ---
Enter the day name: Saturday
Enter the date: 141025
Enter the activity for the day: Shopping
--- Enter details for Day 7 ---
Enter the day name: Sunday
Enter the date: 151025
Enter the activity for the day: Playing

Weekly Activity Details:
Day 1: Monday
Date: 101025

Activity: Reading
Day 2: Tuesday
Date: 111025

Activity: Coding
Day 3: Wednesday
Date: 121025

Activity: Painting
Day 4: Thursday
Date: 131025
Activity: Watching Movie

Day 5: Friday
Date: 141025
Activity: Reading

Day 6: Saturday
Date: 141025
Activity: Shopping

Day 7: Sunday
Date: 151025
Activity: Playing

ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$ ^C
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$

Viva Question:

1. What is the main purpose of this program?

→ To create a weekly calendar with day names, dates, and activities, and display the weekly schedule.

2. What data structure is used to represent a day?

→ A structure struct Day with three fields: name (string), date (integer), and activity (string).

3. Why are name and activity fields dynamically allocated?

→ To allow flexible memory allocation for strings and avoid fixed-size limitations.

4. What is the purpose of the create() function?

→ To read the details of a single day and return a Day structure with dynamically allocated memory.

5. How does the read() function work?

→ It loops through the calendar array and calls create() for each day to input details.

6. How does the display() function work?

→ It prints the day name, date, and activity for all elements of the calendar array.

7. Why is it important to free the dynamically allocated memory?

→ To p revent memory leaks, which can occur if allocated memory is not released after use.

8. What would happen if we forget to use free()?

→ The program would consume memory unnecessarily and could lead to memory exhaustion in long-running programs.

9. Can this program handle more than 7 days?

→ Yes, by changing week Size and ensuring the array is resized or dynamically allocated.

10. What is the advantage of using dynamic allocation for strings over fixed arrays?

→It allows storing strings of varying lengths efficiently and reduces unused memory.

EXPERIMENT - 02

Develop a program in C for the following operations on Strings

a. Read a Main String (STR), a Pattern String (PAT) and a Replace String (REP).
b. Perform Pattern Matching Operation: Find and Replace all occurrences of PAT in STR with REP if PAT exists in STR. Repost suitable messages in case PAT does not exist in STR.

Support the program with functions for each of the above operations. Don't use built-in functions.

ABOUT THE EXPERIMENT:

Strings are actually one-dimensional array of characters terminated by a **null** character '\0'. Thus a null- terminated string contains the characters that comprise the string followed by a **null**.

The following declaration and initialization create a string consisting of the word "Hello". To hold the null character at the end of the array, the size of the character array containing the string is one more than the number of characters in the word "Hello."

char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};

If you follow the rule of array initialization then you can write the above statement as follows:

char greeting[] = "Hello";

C language supports a wide range of built-in functions that manipulate null-terminated strings as follows:

strcpy(s1, s2); Copies string s2 into string s1.
strcat(s1, s2); Concatenates string s2 onto the
end of string s1.strlen(s1); returns the length of string s1.
strcmp(s1, s2); Returns 0 if s1 and s2 are the same; less than 0 if
s1<s2; greater
than 0 if s1>s2.S
strchr(s1, ch); Returns a pointer to the first occurrence of character ch in
string s1. strstr(s1, s2); Returns a pointer to the first occurrence of string s2 in
string s1.

PROGRAM CODE:

```
#include <stdio.h>
#include <string.h>

char str[100], pat[50], rep[50], ans[200];
int i, j, k, flag = 0;

void stringmatch()
{
  i = j = 0;
  while (str[i] != '\0')
   {
     int m = i, n = 0;
     while (str[m] == pat[n] && pat[n] != '\0')
      {
```

```
            m++; n++;
          }
        if (pat[n] == '\0')
          {
              // Full match found flag = 1;
              for (k = 0; rep[k] != '\0'; k++)
              ans[j++] = rep[k];
              i = m; // Skip the matched part
          }
        else
          {
              ans[j++] = str[i++];
          }
    }
    ans[j] = '\0';
    }

      int main()
      {
              printf("Enter the main string:\n");
              scanf(" %[^\n]", str);
              printf("\nEnter the Pattern string:\n");
              scanf("%s", pat);
              printf("\nEnter the Replace string:\n");
              scanf("%s", rep);
              stringmatch();
              if (flag)
                  printf("\nThe resultant string is:\n%s\n", ans);
              else
                  printf("\nPattern string NOT found\n");
              return 0;
      }
```

SAMPLE OUTPUT:

```
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$ gcc p2.c
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$ ./a.out

    Enter the main string:

    this is the program

    Enter the Pattern string:

    program

    Enter the Replace string:

    code program

    The resultant string is:

    this is the code
```

VIVA QUESTION:

1. What is the main purpose of this program?
→ To find all occurrences of a pattern string in a main string and replace them with another string.

2. What is the role of the flag variable?
→ It indicates whether the pattern was found in the main string.

3. Which arrays are used and why?
→ str[] → main string, pat[] → pattern string, rep[] → replacement string, ans[]
→ stores the result after replacement.

4. How does the program handle pattern matching?
→ It scans str[] character by character and compares it with pat[]. On a match, it copies rep[] to ans[].

5. What happens if the pattern does not exist in the main string?
→ The program prints "Pattern string NOT found".

6. Can the program handle multiple occurrences of the pattern?
→ Yes, it replaces all occurrences of the pattern in the main string.

7. Does this program handle overlapping patterns?
→ No, overlapping occurrences of PAT may not be replaced correctly.

8. Why do we use multiple index variables (i, j, c, m)?
→ To separately track positions in the pattern, result, and main string to manage copying and replacement efficiently.

9. What is the time complexity of the pattern matching algorithm?
→ O(n*m), where n = length of main string, m = length of pattern.

10. How can this program be modified for case-insensitive pattern matching?
→ Convert both STR and PAT to the same case (e.g., all lowercase) before performing matching.

EXPERIMENT - 03

Develop a menu driven program in C for the following operations on **STACK** of integers (Array implementation of stack with maximum size **MAX**)

a. Push an element on to stack
b. Pop an element from stack.
c. Demonstrate how stack can be used to check palindrome.
d. Demonstrate Overflow and Underflow situations on stack.
e. Display the status of stack.
f. Exit.

Support the program with appropriate functions for each of the above operations.

ABOUT THE EXPERIMENT:

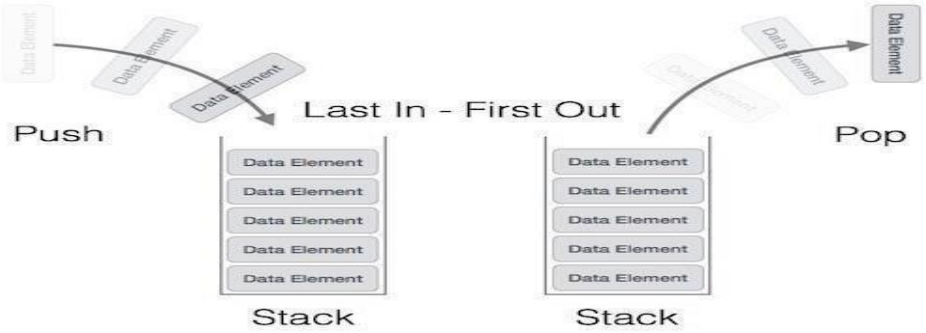
A stack is an abstract data type (ADT), commonly used in most programming languages. It is named stack as it behaves like a real-world stack, for example − deck of cards or pile of plates etc.




A real-world stack allows operations at one end only. For example, we can place or remove a card or plate from top of the stack only. Likewise, Stack ADT allows all data operations at one end only. At any given time, we can only access the top element of a stack.

This feature makes it LIFO data structure. LIFO stands for Last-in-first-out. Here, the element which is placed (inserted or added) last is accessed first. In stack terminology, insertion operation is called **PUSH** operation and removal operation is called **POP** operation.

Below given diagram tries to depict a stack and its operations −



A stack can be implemented by means of Array, Structure, Pointer and Linked-List. Stack can either bea fixed size one or it may have a sense of dynamic resizing. Here, we are going to implement stack using arrays which makes it a fixed size stack implementation.

Basic Operations:

- **push()** - pushing (storing) an element on the stack.
- **pop()** - removing (accessing) an element from the stack.

To use a stack efficiently we need to check status of stack as well. For the same purpose, the following functionality is added to stacks;

- . **isFull()** − check if stack is full.

- **isEmpty()**-check if stack is empty

PROGRAM CODE

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#define max 5
int st[max], top=-1;

void push(int item)
{
        if(top==max-1)
        {
          printf("Stack overflow\n");
          return ;
        }
        top=top+1;
        st[top]=item;
}

int pop()
{
        if(top == -1)
        {
                printf("Stack underflow\n");
                return 0;
        }
        int poppeditem=st[top];
        top=top-1;
        return poppeditem;
}

void display()
{
   int i;
   if (top == -1)
   {
        printf("Stack Empty\n");
        return;
   }
  for (i = top; i >= 0; i--)
     printf("%d", st[i]);
}
void palin()
{
   int i;
   char p[100];
   top = -1;
   printf("Enter a number to check for palindrome\n");
   scanf("%s", p);
   for (i = 0; i < strlen(p); i++)
   push(p[i] - '0');
   for (i = 0; i < strlen(p) / 2; i++)
   if ((p[i] - '0') != pop())
```

```
        {
           printf("***The number is not palindrome***");
           return;
        }
       printf("***The number is a palindrome***");
}

void main()
{
        int ch, k, item;
        while(1)
          {
                printf("MAIN MENU\n");
                printf(" 1:Push\n 2:Pop\n 3:Display\n 4:Palindrome\n 5:Exit\n");
                printf("Enter your choice\n");
                scanf("%d",&ch);
                switch(ch)
                 {
                        case 1:
                                printf("Enter an item to push\n");
                                scanf("%d",&item);
                                push(item);
                                break;
                        case 2: k=pop()
                                if(k)
                                printf("popped element is %d\n",k);
                                break;
                        case 3: display();
                                break;
                        case 4: palin();
                                break;
                        default:  exit(0);

                }
          }
}
```

SAMPLE OUTPUT:

ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$ gcc p3.c
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$ ./a.out

```
--- MAIN MENU ---
1: Push
2: Pop
3: Display
4: Palindrome Check
5: Exit
Enter your choice: 3 Stack Empty
--- MAIN MENU ---
1: Push
2: Pop
3: Display
4: Palindrome Check
5: Exit
Enter your choice: 1 Enter an item to push: 10

--- MAIN MENU ---
1: Push
2: Pop
3: Display
4: Palindrome Check
5: Exit
Enter your choice: 1 Enter an item to push: 20

--- MAIN MENU ---
1: Push
2: Pop
3: Display
4: Palindrome Check
5: Exit
Enter your choice: 1 Enter an item to push: 30

--- MAIN MENU ---
1: Push
2: Pop
3: Display
4: Palindrome Check
5: Exit
Enter your choice: 1 Enter an item to push: 40
```

```
--- MAIN MENU ---
1: Push
2: Pop
3: Display
4: Palindrome Check
5: Exit
Enter your choice: 1 Enter an item to push: 50

--- MAIN MENU ---
1: Push
2: Pop
3: Display
4: Palindrome Check
5: Exit
Enter your choice: 1 Enter an item to push: 60

Stack overflow

--- MAIN MENU ---
1: Push
2: Pop
3: Display
4: Palindrome Check
5: Exit
Enter your choice: 3 Stack elements:
50 40 30 20 10

--- MAIN MENU ---
1: Push
2: Pop
3: Display
4: Palindrome Check
5: Exit
Enter your choice: 2 Popped element is 50
```

--- MAIN MENU --- 1: Push 2: Pop 3: Display 4: Palindrome Check 5: Exit Enter your choice: 2 Popped element is 40 --- MAIN MENU --- 1: Push 2: Pop 3: Display 4: Palindrome Check 5: Exit Enter your choice: 2 Popped element is 30 --- MAIN MENU --- 1: Push 2: Pop 3: Display 4: Palindrome Check 5: Exit Enter your choice: 2 Popped element is 20	--- MAIN MENU --- 1: Push 2: Pop 3: Display 4: Palindrome Check 5: Exit Enter your choice: 2 Popped element is 10 --- MAIN MENU --- 1: Push 2: Pop 3: Display 4: Palindrome Check 5: Exit Enter your choice: 2 Stack underflow --- MAIN MENU --- 1: Push 2: Pop 3: Display 4: Palindrome Check 5: Exit ubuntu@ubuntu-VirtualBox:~/DSALAB$

VIVA QUESTION:

1. What is a stack and what is its principle of operation?

→ A stack is a linear data structure that follows LIFO (Last In First Out) principle.

2. How is the stack implemented in this program?

→ Using an integer array st[max] with a top pointer to track the last inserted element.

3. What happens when you try to push an element onto a full stack?

→ The program prints Stack Overflow and does not insert the element.

4. What happens when you try to pop an element from an empty stack?

→ The program prints Stack Underflow and returns 0.

5. How does the program check for a palindrome?

→ It pushes each digit of the number onto the stack, then pops and compares the first half with the second half.

6. Why is a stack suitable for palindrome checking?

→ Because stacks reverse the order of elements automatically, which helps in comparing mirrored positions.

7. What is the time complexity of push and pop operations?

→ Both are O(1) since only the top pointer is incremented/decremented.

8. How is the stack displayed?

→ By iterating from top to 0 and printing each element, showing the current stack status.

9. How can the program demonstrate overflow and underflow?

→ By trying to push elements beyond max size (overflow) or pop from an empty stack (underflow).

10. Can this stack handle non-integer data?

→ Not in this implementation; it only supports integers. For other types, a generic or pointer-based stack is needed.

EXPERIMENT - 04

Develop a Program in C for converting an Infix Expression to Postfix Expression. Program should support for both parenthesized and free parenthesized expressions with the operators: **+, -, *, /, %(Remainder), ^ (Power)** and **alphanumeric** operands**.**

ABOUT THE EXPERIMENT:

Infix: Operators are written in-between their operands. Ex: X+ Y
Prefix: Operators are written before their operands. Ex: +X Y
Postfix: Operators are written after their operands. Ex: XY+

Examples of Infix, Prefix, and Postfix

Infix Expression	Prefix Expression	Postfix Expression
A + B	+ A B	A B +
A + B * C	+ A * B C	A B C * +

Convert Infix expression to postfix form: (A+B^D)/(E-F)+G

Symbol Scanned	STACK	Expression P
	(	
(	((	
A	((	A
+	((+	A
B	((+	A B
^	((+ ^	A B
D	((+ ^	A B D
)	(	A B D ^ +
/	(/	A B D ^ +
(	(/ (	A B D ^ +
E	(/ (	A B D ^ + E
-	(/ (-	A B D ^ + E
F	(/ (-	A B D ^ + E
)	(/	A B D ^ + E -
+	(+	A B D ^ + E - /
G	(+	A B D ^ + E - / G
)		A B D ^ + E - / G +

PROGRAM CODE:

```
#include <stdio.h>
#include <ctype.h>

char stack[100];
int top = -1;

void push(char x) {
   stack[++top] = x;
}

char pop() {
   if (top == -1)
      return -1;
   else
      return stack[top--];
}

int priority(char x) {
   if (x == '(')
      return 0;
   if (x == '+' || x == '-')
      return 1;
   if (x == '*' || x == '/')
      return 2;
   if (x == '^')
      return 3;  // highest precedence
   return 0;
}

// check associativity: left(0) or right(1)
int isRightAssociative(char x) {
   if (x == '^')
      return 1;  // ^ is right associative
   return 0;       // others are left associative
}

int main() {
   char exp[100];
   char *e, x;

   printf("Enter the expression : ");
   scanf("%s", exp);
   printf("\n");

   e = exp;
   while (*e != '\0') {
      if (isalnum(*e)) {
         printf("%c ", *e);
      }
      else if (*e == '(') {
```

```
            push(*e);
        }
        else if (*e == ')') {
            while ((x = pop()) != '(')
                printf("%c ", x);
        }
        else {
                    while (top != -1 &&((priority(stack[top])> priority(*e)) ||
                    (priority(stack[top]) == priority(*e) && !isRightAssociative(*e))))
                  {
                   printf("%c ", pop());
                   }
            push(*e);
        }
        e++;
    }

    while (top != -1) {
        printf("%c ", pop());
    }

    return 0;
}
```

SAMPLE OUTPUT:

```
ubuntu@ubuntu-VirtualBox:~/DSALAB$ touch p4.c
ubuntu@ubuntu-VirtualBox:~/DSALAB$ gcc p4.c
ubuntu@ubuntu-VirtualBox:~/DSALAB$ ./a.out
Enter the expression : (a+(b-c)*d)

a b c - d * +
ubuntu@ubuntu-VirtualBox:~/DSALAB$
```

VIVA QUESTION:

1. Why do we use a stack in infix to postfix conversion?

→ Stack temporarily stores operators until their precedence allows them to be printed.

2. What is the precedence of operators in this program?

→ * / % = 2, + - = 1, (= 0.

3. How are parentheses handled?

→ (→ push;) → pop until (is encountered.

4. Can this program handle multi-character operands like AB?

→ Not currently; it handles only single-character operands.

5. What is the time complexity?

→ O(n), where n = length of infix expression.

6. What is the difference between infix, postfix, and prefix?

○ Infix: Operator between operands (A + B)

○ Postfix: Operator after operands (A B +)

○ Prefix: Operator before operands (+ A B)

7. Why is precedence important?

→ Ensures the correct order of operations in the postfix expression.

8. What happens if the input expression is invalid?

→ The program may produce incorrect output or pop an empty stack (needs error handling).

9. Can this program evaluate the postfix after conversion?

→ Yes, by using a postfix evaluation stack program.

10. How would you modify it for exponent ^ or modulo %?

→ Update priority() function to give ^ higher precedence (e.g., 3), % same as * /.

EXPERIMENT - 05

Develop a Program in C for the following Stack Applications

a. Evaluation of **Suffix expression** with single digit operands and operators: **+, -, *, /, %,** ^

b. Solving **Tower of Hanoi** problem with **n** disks.

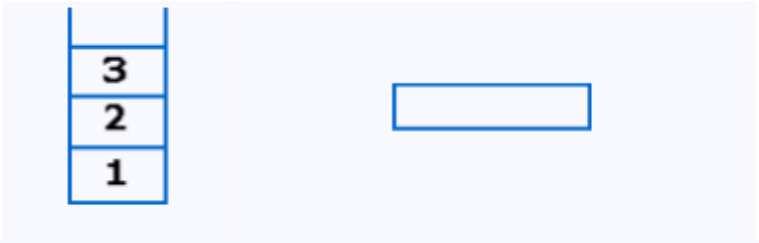
ABOUT THE EXPERIMENT:

a. Evaluation of Suffix expression with single digit operands and operators: +, -, *, /, %, ^

Postfix/Suffix Expression: Operators are written after their operands. Ex: XY+

In normal algebra we use the infix notation like **a+b*c**. The corresponding postfix notation is **abc*+ Example: Postfix String: 123*+4-**

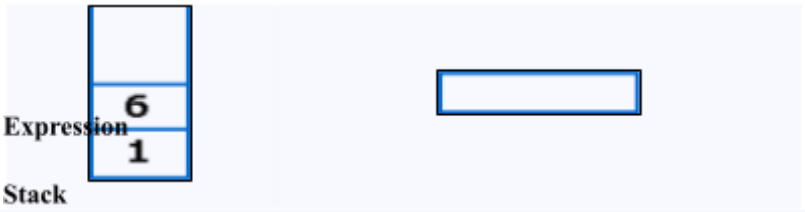
Initially the Stack is empty. Now, the first three characters scanned are 1,2 and 3, which are operands. Thus they will be pushed into the stack in that order.



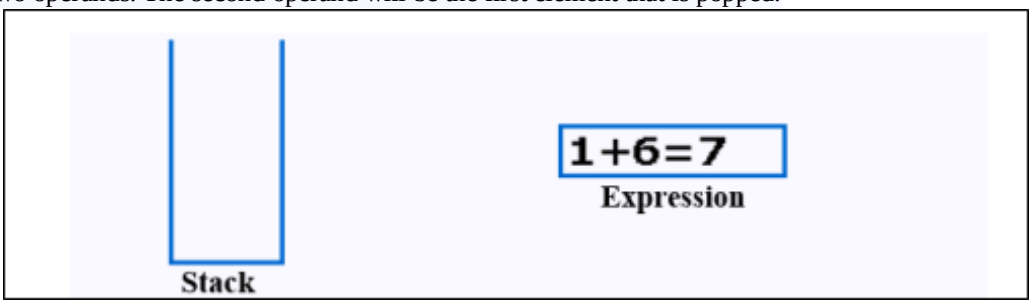
Next character scanned is "*", which is an operator. Thus, we pop the top two elements from the stack and perform the "*" operation with the two operands. The second operand will be the first element that is popped.



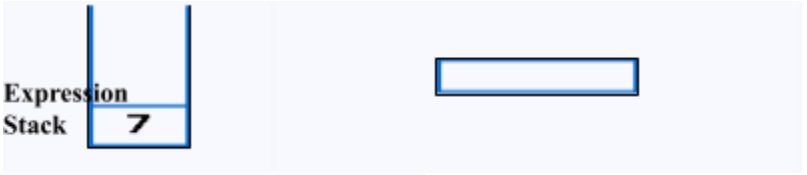
The value of the expression(2*3) that has been evaluated(6) is pushed into the stack.



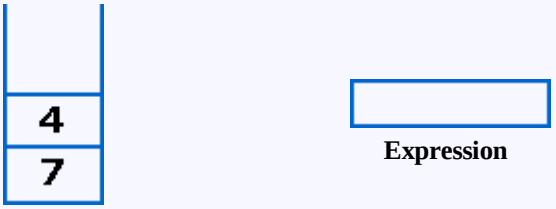
Next character scanned is "+", which is an operator. Thus, we pop the top two elements from the stack and perform the "+" operation with the two operands. The second operand will be the first element that is popped.



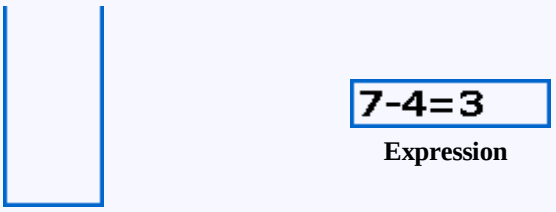
The value of the expression(1+6) that has been evaluated(7) is pushed into the stack.



Next character scanned is "4", which is added to the stack.



Next character scanned is "-", which is an operator. Thus, we pop the top two elements from the stack and perform the "-" operation with the two operands. The second operand will be the first element that is popped.



The value of the expression(7-4) that has been evaluated(3) is pushed into the stack.



Now, since all the characters are scanned, the remaining element in the stack (there will be only one element in the stack) will be returned.

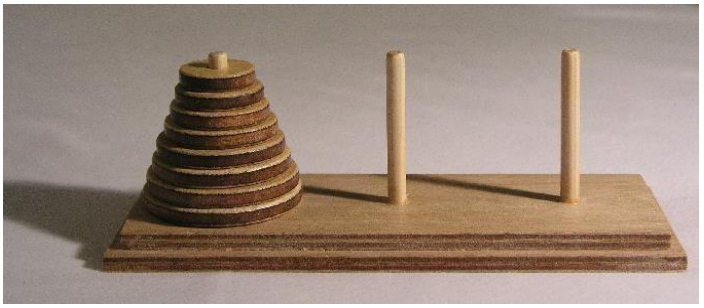
End result:

Postfix String: 123*+4-
Result: 3

The **Tower of Hanoi** is a mathematical game or puzzle. It consists of three rods, and a number of disks of different sizes which can slide onto any rod. The puzzle starts with the disks in a neat stack in ascending order of size on one rod, the smallest at the top, thus making a conical shape.
The objective of the puzzle is to move the entire stack to another rod, obeying the following simple rules:

- Only one disk can be moved at a time.
- Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack
  i.e. a disk can only be moved if it is the uppermost disk on a stack.
- No disk may be placed on top of a smaller disk.

With three disks, the puzzle can be solved in seven moves. The minimum number of moves required to solve a Tower of Hanoi puzzle is $2^n - 1$, where *n* is the number of disks.



PROGRAM CODE:

```
#include <stdio.h>
 #include <math.h>
#include<string.h>
#include <ctype.h> // for isdigit()

double compute (char symbol, double op1, double op2)
{
      switch (symbol)
       {
              case '+': return op1 + op2;
              case '-': return op1 - op2;
              case '*': return op1 * op2;
              case '/': return op1 / op2;
              case '$':
              case '^': return pow(op1, op2);
              default:  return 0;
       }
}

int main()
{
       double s[20], res, op1, op2;
        int top = -1, i;
       char postfix[20], symbol;

       printf("\nEnter the postfix expression:\n");
       scanf("%s", postfix);

       for (i = 0; i < strlen(postfix); i++)
        {
            symbol = postfix[i];
```

```
                if (isdigit(symbol))
                 {
                        s[++top] = symbol - '0'; // convert char to int then to
                 }
                 else
                 {
                        op2 = s[top--];
                        op1 = s[top--];
                        res = compute (symbol, op1, op2);
                        s[++top] = res;
                 }
            }

        res = s[top--];         // Final result
        printf("\nThe result is: %f\n", res);
        return 0;
    }
```

SAMPLOUTPUT:

```
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$ gcc p5.c -lm
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$ ./a.out

Enter the postfix expression:
34+2*4-

The result is: 10.000000
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$  ./a.out

Enter the postfix expression:
34$8*

The result is: 648.000000
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$
```

PROGRAM 5B:

```
#include <stdio.h>
#include <math.h>   // for pow()

void tower(int n, char source, char temp, char destination) {
   if (n == 0)
      return;

   tower(n - 1, source, destination, temp);
   printf("Move disc %d from %c to %c\n", n, source, destination);
   tower(n - 1, temp, source, destination);
}

int main()
 {
   int n;

   printf("Enter the number of discs: ");
   scanf("%d", &n);

   printf("\nSteps:\n");
   tower(n, 'A', 'B', 'C');

   printf("\nTotal Number of moves are: %d\n", (int)pow(2, n) - 1);

   return 0;
}
```

SAMPLE OUTPUT:

```
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_progra m_dsa$ gcc p5b.c -lm
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$ ./a.out

 Enter the number of discs: 3

Steps:
Move disc 1 from A to C
Move disc 2 from A to B
Move disc 1 from C to B
Move disc 3 from A to C
Move disc 1 from B to A
Move disc 2 from B to C
Move disc 1 from A to C

Total Number of moves are: 7

ubuntu@ubuntu-Virtua lBox:~/DSALAB/all_p rogram_dsa$
```

VIVA QUESTION:

1. Why use a stack for postfix evaluation?
→ Stack stores operands and allows easy retrieval of the last two operands for each operator.
2. What is the time complexity?
→ O(n), where n = length of postfix expression.
3. Why convert char to int using symbol - '0'?
→ Because input is character, we need numeric value for computation.
4. Can this program handle multi-digit operands?
→ Not in current form; needs modifications with token parsing.
5. Why recursion is used in Tower of Hanoi?
→ Because each step can be broken down into smaller similar sub-problems.
6. Total moves formula?
→ 2^n - 1.
7. Time complexity?
→ O(2^n), exponential growth.
8. Can we solve Tower of Hanoi using stacks?
→ Yes, by simulating moves iteratively using 3 stacks representing rods.

EXPERIMENT - 06

Develop a menu driven Program in C for the following operations on **Circular QUEUE** of Characters (Array Implementation of Queue with maximum size **MAX**)

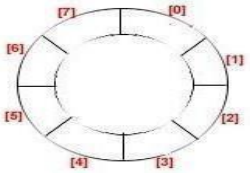
a. Insert an Element on to Circular QUEUE
b. Delete an Element from Circular QUEUE
c. Demonstrate ***Overflow*** and ***Underflow*** situations on Circular QUEUE
d. Display the status of Circular QUEUE
e. Exit

Support the program with appropriate functions for each of the above operations

ABOUT THE EXPERIMENT:

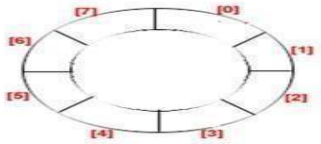
Circular queue is a linear data structure. It follows FIFO principle. In **circular queue** the last node is connected back to the first node to make a **circle**. **Circular** linked list fallow the First In First Out principle. Elements are added at the rear end and the elements are deleted at front end of the **queue**. The queue is considered as a circular queue when the positions 0 and MAX-1 are adjacent. Any position before front is also after rear.

A circular queue looks like

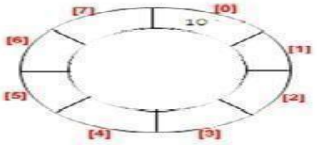


Consider the example with Circular Queue implementation:

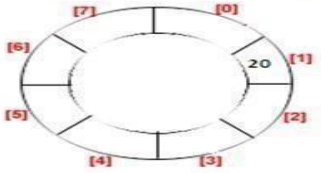
1) Initially: **Front = 0** and **rear = -1**



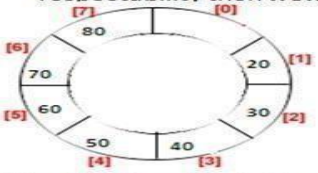
2) Add item 10 then **front = 0** and **rear =0**.



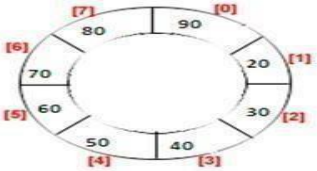
3) Now delete one item then **front = 1** and **rear =1**.



4) Like this now insert 30, 40, and 50,50,70,80 respectability then **front =1** and **rear = 7.**



5) Now in case of linear queue, we can not access 0 block for insertion but in circular queue next item will be inserted of 0 block then **front =0** and **rear = 0.**



PROGRAM CODE:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 4
int ch, front = 0, rear = -1, count = 0;

int  q[MAX], item;
void insert(int  item)
{
          if (count == MAX)
           {
              printf("\nQueue is Full");
              return;
           }
         rear = (rear + 1) % MAX;
         q[rear] = item;
         count++;
}

void del()
{
           if (count == 0)
            {
               printf("\nQueue is Empty");
               return;
            }
          item = q[front];
          printf("\nDeleted item is: %d", item);
          front = (front + 1) % MAX;
          count--;
}

void display()
 {
     if (count == 0)
      {
        printf("\nQueue is Empty"); return;
      }
  printf("\nThe contents of the queue are:\n");
 int i, f = front;
 for (i = 0; i < count; i++)
 {
   printf("%d\t", q[f]);
   f = (f + 1) % MAX;
 }
printf("\n");
}

int main()
{
   while (1)
    {
          printf("\n1. Insert\n2. Delete\n3. Display\n4. Exit");
          printf("\nEnter the choice: ");
```

```
            scanf("%d", &ch);
            switch (ch)
                {
                    case 1: printf("\nEnter the character / item to be inserted: ");
                            scanf(" %d", &item);
                            break;
                    case 2: del();
                            break;
                    case 3: display();
                            break;
                    case 4: exit(0);
                    default: printf("\nInvalid choice!");
                }
        }
        return 0;
    }
```

SAMPLE POUTPUT:

ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$ gcc p6.c
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$ ./a.out

```
Insert
Delete
Display
Exit
Enter the choice: 3

Queue is Empty
Insert
Delete
Display
Exit
Enter the choice: 1

Enter the item to be inserted: 10

Insert
Delete
Display
Exit
Enter the choice: 1

Enter the item to be inserted: 20

Insert
Delete
Display
Exit
Enter the choice: 1

Enter the item to be inserted: 30

Insert
Delete
Display
Exit
Enter the choice: 1
```

```
Enter the item to be inserted: 40

Insert
Delete
Display
Exit
Enter the choice: 1
Enter the item to be inserted: 50
Queue is Full
Insert
Delete
Display
Exit
Enter the choice: 1
Enter the item to be inserted: 60
Queue is Full
Insert
Delete
Display
Exit
Enter the choice: 3

The contents of the queue are:
10  20  30      40

Insert
Delete
Display
Exit
Enter the choice: 2

Deleted item is: 10
Insert
Delete
Display
Exit
```

Enter the choice: 2

Deleted item is: 20
1. Insert
2. Delete
3. Display
4. Exit
Enter the choice: 2

Deleted item is: 30
1. Insert
2. Delete
3. Display
4. Exit
Enter the choice: 2

Deleted item is: 40
1. Insert
2. Delete
3. Display
4. Exit
Enter the choice: 2

Queue is Empty
1. Insert
2. Delete
3. Display
4. Exit
Enter the choice: 3

Queue is Empty
1. Insert
2. Delete
3. Display
4. Exit
Enter the choice: 4
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$

VIVA QUESTION:

1. What is a Circular Queue?

→A Circular Queue is a linear data structure that follows FIFO (First In First Out) but connects the end of the queue back to the front, forming a circle. This helps in efficient use of memory.

2.Why do we use a Circular Queue instead of a normal Queue?

→In a normal queue, once the rear reaches the end, no more insertions are possible even if there are empty spaces in front. A circular queue reuses the freed space using the formula: rear = (rear + 1) % MAX.

3.What are the front and rear pointers used for?

- front → points to the position of the first element (for deletion).
- rear → points to the position of the last inserted element (for insertion).

4.How do you check if the circular queue is full or empty?

- Full Condition: count == MAX
- Empty Condition: count == 0

5.What is the purpose of the modulo operator (%) in this program?

→It ensures wrap-around indexing, i.e., after the last index (MAX-1), the next element goes back to index 0.

6.Why is count used in this program?

→count keeps track of the number of elements in the queue and helps in checking whether the queue is full or empty easily.

7.What happens when you try to delete from an empty queue?

→The program prints: Queue is Empty and does not perform deletion, preventing underflow.

8.What happens when you try to insert into a full queue?

→The program prints: Queue is Full and does not insert the new element, preventing overflow.

9.What is the time complexity of insertion and deletion in a circular queue?

→Both Insertion and Deletion take O(1) time because no shifting of elements is needed.

10.What are the real-life applications of Circular Queues?

- CPU process scheduling (Round Robin).
- Buffer management (I/O buffers, keyboard buffers).
- Traffic signal control systems.
- Printer job scheduling.

EXPERIMENT - 07

Develop a menu driven Program in C for the following operations on
Singly Linked List (SLL) of Student Data with the fields: ***USN, Name, Branch, Sem, PhNo***

a. Create a **SLL** of **N** Students Data by using ***front insertion***.
b. Display the status of **SLL** and count the number of nodes in it
c. Perform Insertion and Deletion at End of **SLL**
d. Perform Insertion and Deletion at Front of **SLL**

ABOUT THE EXPERIMENT:

Linked List is a linear data structure and it is very common data structure which consists of group of nodes in a sequence which is divided in two parts. Each node consists of its own data and the address of the next node and forms a chain. Linked Lists are used to create trees and graphs.



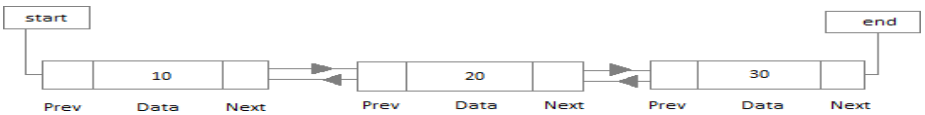
- They are a dynamic in nature which allocates the memory when required.
- Insertion and deletion operations can be easily implemented.
- Stacks and queues can be easily executed.
- Linked List reduces the access time.
- Linked lists are used to implement stacks, queues, graphs, etc.
- Linked lists let you insert elements at the beginning and end of the list.
- In Linked Lists we don't need to know the size in advance.

Types of Linked List:

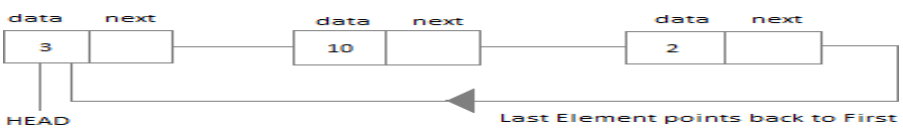
Singly Linked List: Singly linked lists contain nodes which have a data part as well as an address part i.e. next,which points to the next node in sequence of nodes. The operations we can perform on singly linked lists are insertion, deletion and traversal.



Doubly Linked List: In a doubly linked list, each node contains two links the first link points to the previous node and the next link points to the next node in the sequence.



Circular Linked List: In the circular linked list the last node of the list contains the address of the first node and forms a circular chain.



PROGRAM CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Structure Definition
struct Student
{
    char USN[15];
    char Name[25];
    char Branch[10];
    int Sem;
    char PhNo[15];
    struct Student *next;
};
typedef struct Student Node;
Node *head = NULL;
// Function Prototypes
void create();
void display();
void insert_front();
void insert_end();
void delete_front();
void delete_end();
int count_nodes();
// Function to create SLL using front insertion
void create()
{
    int n, i;
    printf("Enter number of students: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("\nEnter details of student %d:\n", i + 1);
        insert_front();
    }
}
```

```
// Function to insert node at front
void insert_front()
{
    Node *temp = (Node *)malloc(sizeof(Node));
    printf("Enter USN: ");
    scanf("%s", temp->USN);
    printf("Enter Name: ");
    scanf("%s", temp->Name);
    printf("Enter Branch: ");
    scanf("%s", temp->Branch);
    printf("Enter Sem: ");
    scanf("%d", &temp->Sem);
    printf("Enter Phone No: ");
    scanf("%s", temp->PhNo);
    temp->next = head;
    head = temp;
    printf("Student inserted at FRONT successfully.\n");
}
// Function to insert node at end
void insert_end()
{
    Node *temp = (Node *)malloc(sizeof(Node));
    printf("Enter USN: ");
    scanf("%s", temp->USN);
    printf("Enter Name: ");
    scanf("%s", temp->Name);
    printf("Enter Branch: ");
    scanf("%s", temp->Branch);
    printf("Enter Sem: ");
    scanf("%d", &temp->Sem);
    printf("Enter Phone No: ");
    scanf("%s", temp->PhNo);
    temp->next = NULL;
    if (head == NULL)
 {
        head = temp;
```

```
    }
else
{
    Node *cur = head;
    while (cur->next != NULL)
        cur = cur->next;
    cur->next = temp;
 }
   printf("Student inserted at END successfully.\n");
}
// Function to delete from front
void delete_front()
{
   if (head == NULL)
 {
    printf("List is empty.\n");
    return;
  }
   Node *temp = head;
   printf("Deleted student: %s\n", temp->USN);
   head = head->next;
   free(temp);
}
// Function to delete from end
void delete_end()
 {
   if (head == NULL)
 {
    printf("List is empty.\n");
    return;
   }
   Node *cur = head, *prev = NULL;
   if (head->next == NULL)
{
    printf("Deleted student: %s\n", head->USN);
    free(head);
    head = NULL;
```

```
        return;
    }
    while (cur->next != NULL)
    {
        prev = cur;
        cur = cur->next;
    }
    printf("Deleted student: %s\n", cur->USN);
    prev->next = NULL;
    free(cur);
}
// Display students and count
void display()
{
    Node *temp = head;
    int count = 0;
    if (temp == NULL)
    {
        printf("\nList is empty.\n");
        return;
    }
    printf("\nStudent Details:\n");
    printf("------------------------------------------------------------\n");
    while (temp != NULL)
    {
        printf("USN: %s | Name: %s | Branch: %s | Sem: %d | PhNo: %s\n",
            temp->USN, temp->Name, temp->Branch, temp->Sem, temp->PhNo);
        temp = temp->next;
        count++;
    }
    printf("------------------------------------------------------------\n");
    printf("Total Students: %d\n", count);
}
// Count nodes
int count_nodes()
{
    Node *temp = head;
```

```
    int count = 0;
    while (temp != NULL)
{
        count++;
        temp = temp->next;
    }
    return count;
}
int main()
{
    int choice;
    while (1)
{
        printf("\n========== SINGLY LINKED LIST MENU ==========\n");
        printf("1. Create SLL of Students (Front Insertion)\n");
        printf("2. Display Students & Count\n");
        printf("3. Insert at End\n");
        printf("4. Delete from End\n");
        printf("5. Insert at Front\n");
        printf("6. Delete from Front\n");
        printf("7. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
         {
            case 1: create(); break;
            case 2: display(); break;
            case 3: insert_end(); break;
            case 4: delete_end(); break;
            case 5: insert_front(); break;
            case 6: delete_front(); break;
            case 7: exit(0);
            default: printf("Invalid choice! Try again.\n");
        }
    }
    return 0;
}
```

SAMPLE OUTPUT:

```
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$ gcc p7.c
ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$ ./a.out

========== SINGLY LINKED LIST MENU ==========
1. Create SLL of Students (Front Insertion)
2. Display Students & Count
3. Insert at End
4. Delete from End
5. Insert at Front
6. Delete from Front
7. Exit
Enter your choice: 2
List is empty.

========== SINGLY LINKED LIST MENU ==========
1. Create SLL of Students (Front Insertion)
2. Display Students & Count
3. Insert at End
4. Delete from End
5. Insert at Front
6. Delete from Front
7. Exit
Enter your choice: 1
Enter number of students: 2

Enter details of student 1:
Enter USN: 1EPCS24001
Enter Name: KUMAR
Enter Branch: CSE
Enter Sem: 3
```

Enter Phone No: 9087654321

Student inserted at FRONT successfully.

Enter details of student 2:

Enter USN: 1EPCS24003

Enter Name: RAVI

Enter Branch: CSE

Enter Sem: 3

Enter Phone No: 8096543261

Student inserted at FRONT successfully.

========== SINGLY LINKED LIST MENU ==========

1. Create SLL of Students (Front Insertion)

2. Display Students & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 2

Student Details:

USN: 1EPCS24003 | Name: RAVI | Branch: CSE | Sem: 3 | PhNo: 8096543261

USN: 1EPCS24001 | Name: KUMAR | Branch: CSE | Sem: 3 | PhNo: 9087654321

Total Students: 2

========== SINGLY LINKED LIST MENU ==========

1. Create SLL of Students (Front Insertion)

2. Display Students & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 3

Enter USN: 1EPCS24008

Enter Name: RANI

Enter Branch: CSE

Enter Sem: 3

Enter Phone No: 9078654534

Student inserted at END successfully.

========== SINGLY LINKED LIST MENU ==========

1. Create SLL of Students (Front Insertion)

2. Display Students & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 3

Enter USN: 1EPCS24009

Enter Name: RAJU

Enter Branch: CSE

Enter Sem: 3

Enter Phone No: 9087654323

Student inserted at END successfully.

========== SINGLY LINKED LIST MENU ==========

1. Create SLL of Students (Front Insertion)

2. Display Students & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 2

Student Details:

USN: 1EPCS24003 | Name: RAVI | Branch: CSE | Sem: 3 | PhNo: 8096543261

USN: 1EPCS24001 | Name: KUMAR | Branch: CSE | Sem: 3 | PhNo: 9087654321

USN: 1EPCS24008 | Name: RANI | Branch: CSE | Sem: 3 | PhNo: 9078654534

USN: 1EPCS24009 | Name: RAJU | Branch: CSE | Sem: 3 | PhNo: 9087654323

Total Students: 4

========== SINGLY LINKED LIST MENU ==========

1. Create SLL of Students (Front Insertion)

2. Display Students & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 5

Enter USN: 1EPCS24007

Enter Name: MANJU

Enter Branch: CSE

Enter Sem: 3

Enter Phone No: 9021785634

Student inserted at FRONT successfully.

========== SINGLY LINKED LIST MENU ==========

1. Create SLL of Students (Front Insertion)

2. Display Students & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 2

Student Details:

USN: 1EPCS24007 | Name: MANJU | Branch: CSE | Sem: 3 | PhNo: 9021785634

USN: 1EPCS24003 | Name: RAVI | Branch: CSE | Sem: 3 | PhNo: 8096543261

USN: 1EPCS24001 | Name: KUMAR | Branch: CSE | Sem: 3 | PhNo: 9087654321

USN: 1EPCS24008 | Name: RANI | Branch: CSE | Sem: 3 | PhNo: 9078654534

USN: 1EPCS24009 | Name: RAJU | Branch: CSE | Sem: 3 | PhNo: 9087654323

Total Students: 5

========== SINGLY LINKED LIST MENU ==========

1. Create SLL of Students (Front Insertion)

2. Display Students & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 4

Deleted student: 1EPCS24009

========== SINGLY LINKED LIST MENU ==========

1. Create SLL of Students (Front Insertion)

2. Display Students & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 6

Deleted student: 1EPCS24007

========== SINGLY LINKED LIST MENU ==========

1. Create SLL of Students (Front Insertion)

2. Display Students & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 2

Student Details:

USN: 1EPCS24003 | Name: RAVI | Branch: CSE | Sem: 3 | PhNo: 8096543261

USN: 1EPCS24001 | Name: KUMAR | Branch: CSE | Sem: 3 | PhNo: 9087654321

USN: 1EPCS24008 | Name: RANI | Branch: CSE | Sem: 3 | PhNo: 9078654534

Total Students: 3

========== SINGLY LINKED LIST MENU ==========

1. Create SLL of Students (Front Insertion)

2. Display Students & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 7

ubuntu@ubuntu-VirtualBox:~/DSALAB/all_program_dsa$

VIVA QUESTION:

1.What is a Singly Linked List (SLL)?

→A Singly Linked List is a collection of nodes where each node contains data and a pointer (next) to the next node in the list. The last node points to NULL, indicating the end of the list.

2.What are the advantages of using a Linked List over an Array?

- Dynamic size — can grow or shrink at runtime.
- Easy insertion and deletion (no need to shift elements).
- Memory is used efficiently (no fixed size required).

3.What is the purpose of using malloc() in this program?

- malloc() dynamically allocates memory for a new node during runtime.
- It ensures that nodes are created as needed, rather than reserving space in advance.

4.What are the different insertion operations in this program?

→There are two types of insertions:

- Front Insertion: Adds a new node at the beginning of the list.
- End Insertion: Adds a new node at the end of the list.

5.What are the different deletion operations in this program? Two types of deletions are performed:

- Front Deletion: Removes the first node in the list.
- End Deletion: Removes the last node in the list.

6.How is the count of nodes in the list determined?

→By traversing from the head to the end of the list and counting each node using a loop in the function count_nodes().

7.How can a Singly Linked List be used as a Stack and Queue?

- As a Stack (LIFO): Perform insertion and deletion at the front.
- As a Queue (FIFO): Perform insertion at the end and deletion at the front.

8.What will happen if you try to delete a node from an empty list?

- The program checks if the list is empty (head == NULL) and prints: List is empty.
- No deletion occurs to prevent runtime errors.

9What is the time complexity of insertion and deletion in a Singly Linked List?

- Insertion at front: O(1)
- Insertion at end: O(n)
- Deletion at front: O(1)
- Deletion at end: O(n)

10 What are the applications of Singly Linked Lists?

- Implementation of stacks, queues, and hash tables.
- Dynamic memory allocation and polynomial representation.
- Used in graph and adjacency list representations.

EXPERIMENT - 08

Develop a menu driven Program in C for the following operations on **Doubly Linked List (DLL)** of Employee Data with the fields: ***SSN, Name, Dept, Designation, Sal, PhNo.***

a. Create a **DLL** of **N** Employees Data by using ***end insertion***.

b. Display the status of **DLL** and count the number of nodes in it

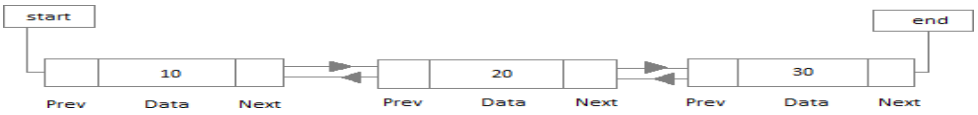
c. Perform Insertion and Deletion at End of **DLL**

d. Perform Insertion and Deletion at Front of **DLL**

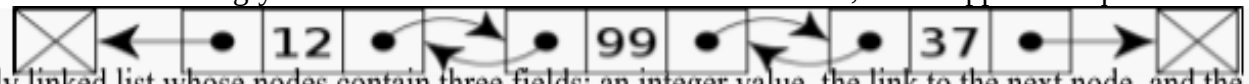
e. Demonstrate how this **DLL** can be used as **Double Ended Queue**

ABOUT THE EXPERIMENT:

Doubly Linked List: In a doubly linked list, each node contains two links the first link points to the previous node and the next link points to the next node in the sequence.



In computer science, a **doubly linked list** is a linked data structure that consists of a set of sequentially linked records called nodes. Each node contains two fields, called *links*, that are references to the previous andto the next node in the sequence of nodes. The beginning and ending nodes' **previous** and **next** links, respectively, point to some kind of terminator, typically a sentinel node or null, to facilitate traversal of the list. If there is only one sentinel node, then the list is circularly linked via the sentinel node. It can be conceptualized as two singly linked lists formed from the same data items, but in opposite sequential orders.



A doubly linked list whose nodes contain three fields: an integer value, the link to the next node, and the link to the previous node.

The two-node links allow traversal of the list in either direction. While adding or removing a node in a doubly linked list requires changing more links than the same operations on a singly linked list, the operations are simpler and potentially more efficient (for nodes other than first nodes) because there is no need to keep track of the previous node during traversal or no need to traverse the list to find the previous node, so that its link can be modified.

Program Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Structure for Employee
struct Employee
{
   char SSN[20];
   char Name[30];
   char Dept[20];
   char Designation[20];
   float Sal;
   char PhNo[15];
   struct Employee *prev, *next;
};
```

```
typedef struct Employee Node;
Node *head = NULL, *tail = NULL;

// Function Prototypes
void create();
void display();
void insert_end();
void insert_front();
void delete_end();
void delete_front();
int count_nodes();

// Function to create DLL using end insertion
void create()
{
   int n, i;
   printf("Enter number of employees: ");
   scanf("%d", &n);
   for (i = 0; i < n; i++)
    {
      printf("\nEnter details of Employee %d:\n", i + 1);
      insert_end();
    }
 }

// Function to insert at end
void insert_end() {
   Node *temp = (Node *)malloc(sizeof(Node));

   printf("Enter SSN: ");
   scanf("%s", temp->SSN);

   printf("Enter Name: ");
   scanf("%s", temp->Name);

   printf("Enter Dept: ");
   scanf("%s", temp->Dept);

   printf("Enter Designation: ");
   scanf("%s", temp->Designation);

   printf("Enter Salary: ");
   scanf("%f", &temp->Sal);

   printf("Enter Phone No: ");
   scanf("%s", temp->PhNo);

   temp->next = NULL;
   temp->prev = NULL;

   if (head == NULL) {
      head = tail = temp;
   }
  else {
      tail->next = temp;
      temp->prev = tail;
```

```
        tail = temp;
    }

    printf("Employee inserted at END successfully.\n");
}

// Function to insert at front
void insert_front() {
    Node *temp = (Node *)malloc(sizeof(Node));

    printf("Enter SSN: ");
    scanf("%s", temp->SSN);

    printf("Enter Name: ");
    scanf("%s", temp->Name);

    printf("Enter Dept: ");
    scanf("%s", temp->Dept);

    printf("Enter Designation: ");
    scanf("%s", temp->Designation);

    printf("Enter Salary: ");
    scanf("%f", &temp->Sal);

    printf("Enter Phone No: ");
    scanf("%s", temp->PhNo);

    temp->next = NULL;
    temp->prev = NULL;

    if (head == NULL) {
        head = tail = temp;
    }
   else {
        temp->next = head;
        head->prev = temp;
        head = temp;
    }

    printf("Employee inserted at FRONT successfully.\n");
}

// Function to delete from end
void delete_end() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    Node *temp = tail;
    printf("Deleted Employee SSN: %s\n", temp->SSN);

    if (head == tail) {
        head = tail = NULL;
    }
```

```
    else
    {
        tail = tail->prev;
        tail->next = NULL;
    }

    free(temp);
}

// Function to delete from front
void delete_front() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    Node *temp = head;
    printf("Deleted Employee SSN: %s\n", temp->SSN);

    if (head == tail) {
        head = tail = NULL;
    } else {
        head = head->next;
        head->prev = NULL;
    }

    free(temp);
}

// Function to display DLL
void display() {
    Node *temp = head;
    int count = 0;

    if (head == NULL) {
        printf("\nList is empty.\n");
        return;
    }

    printf("\nEmployee Details:\n");
    printf("------------------------------------------------------------\n");

    while (temp != NULL) {
        printf("SSN: %s | Name: %s | Dept: %s | Designation: %s | Salary: %.2f | PhNo: %s\n",
               temp->SSN, temp->Name, temp->Dept, temp->Designation, temp->Sal, temp->PhNo);
        temp = temp->next;
        count++;
    }

    printf("------------------------------------------------------------\n");
    printf("Total Employees: %d\n", count);
}

// Function to count nodes
int count_nodes() {
    int count = 0;
```

```
    Node *temp = head;

    while (temp != NULL) {
       count++;
       temp = temp->next;
    }
    return count;
}
int main()
{
    int choice;
    while (1)
    {
       printf("\n========== DOUBLY LINKED LIST MENU ==========\n");
       printf("1. Create DLL of Employees\n");
       printf("2. Display Employees & Count\n");
       printf("3. Insert at End\n");
       printf("4. Delete from End\n");
       printf("5. Insert at Front\n");
       printf("6. Delete from Front\n");
       printf("7. Exit\n");
       printf("Enter your choice: ");
       scanf("%d", &choice);
       switch (choice)
      {
          case 1: create(); break;
          case 2: display(); break;
          case 3: insert_end(); break;
          case 4: delete_end(); break;
          case 5: insert_front(); break;
          case 6: delete_front(); break;
          case 7: exit(0);
          default: printf("Invalid choice! Try again.\n");
       }
    }

    return 0;
}
```

SAMPLE OUTPUT:

```
ubuntu@ubuntu-VirtualBox:~/DSALAB$ gcc p8.c
ubuntu@ubuntu-VirtualBox:~/DSALAB$ ./a.out

========== DOUBLY LINKED LIST MENU ==========
1. Create DLL of Employees
2. Display Employees & Count
3. Insert at End
4. Delete from End
5. Insert at Front
6. Delete from Front
7. Exit
Enter your choice: 2
List is empty.
========== DOUBLY LINKED LIST MENU ==========
1. Create DLL of Employees
2. Display Employees & Count
3. Insert at End
4. Delete from End
5. Insert at Front
6. Delete from Front
7. Exit
Enter your choice: 1
Enter number of employees: 2

Enter details of Employee
1: Enter SSN: 004
Enter Name: kumar
Enter Dept: research
Enter Designation: team_leader
Enter Salary: 3400000
Enter Phone No: 907856432
Employee inserted at END successfully.

Enter details of Employee
2: Enter SSN: 007
Enter Name: rani
Enter Dept: CEO
```

Enter Designation: CEO
Enter Salary: 9000000
Enter Phone No: 8967453423
Employee inserted at END
successfully.

========== DOUBLY LINKED LIST MENU ==========

1. Create DLL of Employees

2. Display Employees & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 2

Employee Details:

SSN: 004 | Name: kumar | Dept: research | Designation: team_leader | Salary: 3400000.00 | PhNo: 907856432

SSN: 007 | Name: rani | Dept: CEO | Designation: CEO | Salary: 9000000.00 | PhNo: 8967453423

Total Employees: 2

========== DOUBLY LINKED LIST MENU ==========

1. Create DLL of Employees

2. Display Employees & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice:
3 Enter SSN: 009
Enter Name: RANI
Enter Dept: coding
Enter Designation: director
Enter Salary: 4566666
Enter Phone No: 9089786732
Employee inserted at END
successfully.

========== DOUBLY LINKED LIST MENU ==========

1. Create DLL of Employees

2. Display Employees & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 2

Employee Details:

SSN: 004 | Name: kumar | Dept: research | Designation: team_leader | Salary: 3400000.00 | PhNo: 907856432

SSN: 007 | Name: rani | Dept: CEO | Designation: CEO | Salary: 9000000.00 | PhNo: 8967453423

SSN: 009 | Name: RANI | Dept: coding | Designation: director | Salary: 4566666.00 | PhNo: 9089786732

Total Employees: 3

========== DOUBLY LINKED LIST MENU ==========

1. Create DLL of Employees

2. Display Employees & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice:
5 Enter SSN: 006
Enter Name: ram
Enter Dept: project
Enter Designation: manager
Enter Salary: 89000
Enter Phone No: 9023452143

Employee inserted at FRONT successfully.

========== DOUBLY LINKED LIST MENU ==========

1. Create DLL of Employees

2. Display Employees & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice:
3 Enter SSN: 004
Enter Name: rani
Enter Dept: project
Enter Designation: director
Enter Salary: 456565655
Enter Phone No:
9078654321
Employee inserted at END successfully.

========== DOUBLY LINKED LIST MENU ==========

1. Create DLL of Employees

2. Display Employees & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 2

Employee Details:

SSN: 006 | Name: ram | Dept: project | Designation: manager | Salary: 89000.00 | PhNo: 9023452143

SSN: 004 | Name: kumar | Dept: research | Designation: team_leader | Salary: 3400000.00 | PhNo: 907856432

SSN: 007 | Name: rani | Dept: CEO | Designation: CEO | Salary: 9000000.00 | PhNo: 8967453423

SSN: 009 | Name: RANI | Dept: coding | Designation: director | Salary: 4566666.00 | PhNo: 9089786732
SSN: 004 | Name: rani | Dept: project | Designation: director | Salary: 456565664.00 | PhNo: 9078654321

Total Employees: 5

========== DOUBLY LINKED LIST MENU ==========

1. Create DLL of Employees

2. Display Employees & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 4

Deleted Employee SSN: 004

========== DOUBLY LINKED LIST MENU ==========

1. Create DLL of Employees

2. Display Employees & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 6
Deleted Employee SSN:
006

========== DOUBLY LINKED LIST MENU ==========

1. Create DLL of Employees

2. Display Employees & Count

3. Insert at End

4. Delete from End

5. Insert at Front

6. Delete from Front

7. Exit

Enter your choice: 7

ubuntu@ubuntu-VirtualBox:~/DSALAB$

VIVA QUESTION:

1. What is a Doubly Linked List (DLL)?

→A Doubly Linked List is a collection of nodes where each node has three fields:

- Data
- A pointer to the next node
- A pointer to the previous node

This allows traversal in both directions.

2.What are the advantages of a Doubly Linked List over a Singly Linked List?

- Can be traversed both forward and backward.
- Easier to delete a node when its address is known.
- Can efficiently implement double-ended queues (DEQueues).

3.What is the purpose of the prev pointer in each node?

→The pre pointer points to the previous node in the list, enabling backward traversal and easier deletion from both ends.

4.How is a node inserted at the end of the DLL?

- Create a new node using malloc().
- Link the new node's prev to the current tail.
- Update the tail's next to point to the new node.
- Finally, make the new node the new tail of the list.

5.How is a node inserted at the front of the DLL?

- Create a new node.
- Link its next to the current head.
- Update the head's prev to the new node.
- Make the new node the new head.

6.How is a node deleted from the end of the DLL?

- Display the SSN of the last node to be deleted.
- Move the tail pointer one step back using tail = tail->prev.
- Free the memory of the deleted node.
- Set tail->next = NULL.

7.How can a DLL be used as a Double-Ended Queue (DEQueue)?

- Insertion/Deletion at Front → acts like Front End operations.
- Insertion/Deletion at End → acts like Rear End operations.

Thus, DLL supports both input-restricted and output-restricted DEQueues.

8.What happens when you delete from an empty DLL?

- The program checks if head == NULL and displays: List is empty.

- This prevents errors such as accessing invalid memory.

9.What is the time complexity of insertion and deletion in DLL?

- At Front or End: O(1)
- At Specific Position (with traversal): O(n)

10.What are the applications of Doubly Linked Lists?

- Implementation of deques, stacks, and undo/redo features.
- Used in browser history navigation and music playlists.
- Useful in memory management and LRU cache algorithms.

EXPERIMENT - 09

Develop a Program in C for the following operations on Singly Circular Linked List (SCLL) with header nodes

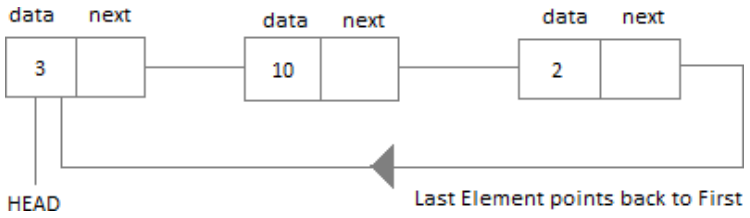
a. Represent and Evaluate a Polynomial P(x, y) = $6x^2y^2 - 4y + 3x^3y + 2xy^5 - 2x$.

Find the sum of two polynomials POLY1(x, y) and POLY2(x, y) and store the result in POLYSUM(x, y)

ABOUT THE EXPERIMENT:

Circular Linked List:

In the circular linked list the last node of the list contains the address of the first node and forms a circular chain.

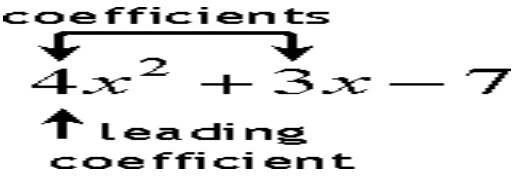


Polynomial:

A **polynomial equation** is an **equation** that can be written in the form. $ax^n + bx^{n-1} + \ldots + rx + s = 0$, where a, b, . . . , r and s are constants. We call the largest exponent of x appearing in a non-zero term of a **polynomial** the degree of that **polynomial**.

As with **polynomials** with one variable, you must pay attention to the rules of exponents and the order of operations so that you correctly **evaluate** an expression with two or more variables. **Evaluate** $x^2 + 3y^3$for x = 7 and y = −2. Substitute the given values for x and y. **Evaluate**$4x^2y – 2xy^2 + x – 7$ for x = 3 and y = −1.

When a term contains both a number and a variable part, the number part is called the "coefficient". The coefficient on the leading term is called the "leading" coefficient.



In the above example, the coefficient of the leading term is 4; the coefficient of the second term is 3; the constant term doesn't have a coefficient.

Here are the steps required for Evaluating Polynomial Functions:

Step 1: Replace each x in the expression with the given value.

Step 2: Use the order of operation to simplify the expression.

Example 1 – Given $f(x) = -2x^2 + 5x - 7$, find $f(3)$.

Step 1: Replace each x in the expression with the given value. In this case, we replace each x with 3.	$f(3) = -2(3)^2 + 5(3) - 7$
Step 2: Use the order of operation to simplify the expression.	$f(3) = -2(9) + 5(3) - 7$ $f(3) = -18 + 15 - 7$ $f(3) = -10$

Here are the steps required for addition of two polynomials.

Step 1

- Arrange the Polynomial in standard form
- Standard form of a polynomial just means that the term with highest degree is first and each of the following terms

Step 2

- Arrange the like terms in columns and add the like terms

Example 1: *Let's find the sum of the following two polynomials*
$(3y^5 - 2y + y^4 + 2y^3 + 5)$ and $(2y^5 + 3y^3 + 2+$

1) Write in Standard form $(3y^5 + y^4 + 2y^3 - 2y + 5) + (2y^5 + 3y^3 + 7y + 2)$

2) Arrange in columns of like terms and then add

$$\begin{array}{r} 3y^5 + y^4 + 2y^3 - 2y + 5 \\ 2y^5 \quad\quad + 3y^3 + 7y + 2 \\ \hline 5y^5 + y^4 + 5y^3 + 5y + 7 \end{array}$$

© mathwarehouse.com

PROGRAM CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

struct node {
   int cf, px, py;          // coefficient and powers of x, y
   struct node *link;
};

typedef struct node *NODE;

// Function to allocate new node
NODE getnode()
{
   NODE x = (NODE)malloc(sizeof(struct node));
   if (x == NULL)
    {
      printf("Insufficient Memory!\n");
      exit(0);
   }
   x->link = x; // self loop initially
   return x;
}

// Insert term at the rear
NODE insert_rear(int cf, int px, int py, NODE head)
{
 NODE temp = getnode();
   temp->cf = cf;
   temp->px = px;
   temp->py = py;

   NODE cur = head;
   while (cur->link != head)
      cur = cur->link;

   cur->link = temp;
   temp->link = head;
   return head;
}

// Display polynomial
void display(NODE head)
 {
NODE temp = head->link;
 if (temp == head)
{
      printf("Polynomial is empty.\n");
      return;
   }
```

```
    while (temp != head)
     {
        printf("%d x^%d y^%d", temp->cf, temp->px, temp->py);
         if (temp->link != head)
        printf(" + ");
        temp = temp->link;
    }
    printf("\n");
}

// Read polynomial
NODE read_poly(NODE head)
{
int cf, px, py, ch;
    do {
        printf("Enter coefficient: ");
        scanf("%d", &cf);
        printf("Enter powers of x and y: ");
        scanf("%d%d", &px, &py);
        head = insert_rear(cf, px, py, head);
        printf("Do you want to add another term? (1-Yes / 0-No): ");
        scanf("%d", &ch);
    } while (ch == 1);
    return head;
}

// Add two polynomials
NODE add_poly(NODE h1, NODE h2, NODE h3)
 {
NODE p1 = h1->link;
    NODE p2 = NULL;
    int found;
    while (p1 != h1)
    {
    found = 0;
        p2 = h2->link;
        while (p2 != h2)
         {
            if (p1->px == p2->px && p1->py == p2->py)
             {
                h3 = insert_rear(p1->cf + p2->cf, p1->px, p1->py, h3);
                p2->cf = 0; // mark used
                found = 1;
                break;
            }
            p2 = p2->link;
        }

        if (!found)
            h3 = insert_rear(p1->cf, p1->px, p1->py, h3);
             p1 = p1->link;
    }
```

```
    // Insert remaining terms from poly2
     p2 = h2->link;
    while (p2 != h2) {
       if (p2->cf != 0)
          h3 = insert_rear(p2->cf, p2->px, p2->py, h3);
           p2 = p2->link;
    }

    return h3;
}

// Evaluate polynomial for given x, y
void evaluate(NODE head)
{
    int x, y;
    double result = 0.0;
    printf("Enter values of x and y: ");
    scanf("%d%d", &x, &y);
    NODE temp = head->link;
     while (temp != head)

     {
       result += temp->cf * pow(x, temp->px) * pow(y, temp->py);
       temp = temp->link;
    }
    printf("Polynomial result = %.2lf\n", result);
}
// Evaluate polynomial for given x, y
void evaluate(NODE head)
 {
    int x, y;
    double result = 0.0;
    printf("Enter values of x and y: ");
    scanf("%d%d", &x, &y);
    NODE temp = head->link;
    while (temp != head)
    {
       result += temp->cf * pow(x, temp->px) * pow(y, temp->py);
       temp = temp->link;
    }
    printf("Polynomial result = %.2lf\n", result);
}
```

```
// Main function
int main()
{
   NODE h1, h2, h3;
   int ch;
   h1 = getnode();
   h2 = getnode();
   h3 = getnode();

   while (1)
   {
      printf("\n--- Polynomial Operations ---\n");
      printf("1. Evaluate Polynomial\n");
      printf("2. Add Two Polynomials\n");
      printf("3. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &ch);

      switch (ch)
      {
       case 1:
            h1 = getnode();
            printf("\nEnter Polynomial:\n");
            h1 = read_poly(h1);
            printf("Entered Polynomial: ");
            display(h1);
            evaluate(h1);
            break;

         case 2:
            h1 = getnode();
            h2 = getnode();
            h3 = getnode();
            printf("\nEnter First Polynomial:\n");
             h1 = read_poly(h1);
            printf("\nEnter Second Polynomial:\n");
            h2 = read_poly(h2);
            printf("\nPOLY1(x,y) = ");
            display(h1);
            printf("POLY2(x,y) = ");
            display(h2);
            h3 = add_poly(h1, h2, h3);
            printf("POLYSUM(x,y) = ");
            display(h3);
            break;

         case 3:
            exit(0);

         default:
            printf("Invalid choice!\n");
      }
   }
   return 0;
}
```

SAMPLE OUTPUT:

```
ubuntu@ubuntu-VirtualBox:~/DSALAB$ gcc p9.c -lm
ubuntu@ubuntu-VirtualBox:~/DSALAB$ ./a.out

--- Polynomial Operations ---
Evaluate Polynomial
Add Two Polynomials
Exit
Enter your choice: 1

Enter Polynomial:
Enter coefficient: 6
Enter powers of x and y: 2 2
Do you want to add another term? (1-Yes / 0-No): 1 Enter coefficient: 3
Enter powers of x and y: 1 1
Do you want to add another term? (1-Yes / 0-No): 0 Entered Polynomial: 6 x^2 y^2 + 3 x^1 y^1
Enter values of x and y: 2 1 Polynomial result = 30.00

--- Polynomial Operations ---
Evaluate Polynomial
Add Two Polynomials
Exit
Enter your choice: 2

Enter First Polynomial:
Enter coefficient: 6
Enter powers of x and y: 2 2
Do you want to add another term? (1-Yes / 0-No): 1 Enter coefficient: 3
Enter powers of x and y: 1 1
Do you want to add another term? (1-Yes / 0-No): 0

Enter Second Polynomial:
Enter coefficient: 4
Enter powers of x and y: 2 2
Do you want to add another term? (1-Yes / 0-No): 1 Enter coefficient: 2
Enter powers of x and y: 1 1
Do you want to add another term? (1-Yes / 0-No): 0

 POLY1(x,y) = 6 x^2 y^2 + 3 x^1 y^1
POLY2(x,y) = 4 x^2 y^2 + 2 x^1 y^1
POLYSUM(x,y) = 10 x^2 y^2 + 5 x^1 y^1

--- Polynomial Operations ---
Evaluate Polynomial
Add Two Polynomials
Exit
Enter your choice: 3
ubuntu@ubuntu-VirtualBox:~/DSALAB$ ^C
ubuntu@ubuntu-VirtualBox:~/DSALAB$
```

VIVA QUESTION:

1.What is a circular linked list?

→A circular linked list is a linked list in which the last node's link points back to the head node instead of NULL, forming a circle.

2.Why do we use a circular linked list in this polynomial program?

→Because it allows easy traversal from any node back to the head, helping to represent the polynomial as a circular structure where the end connects to the beginning useful when repeatedly traversing terms.

3.What is the purpose of the getnode() function?

→getnode() dynamically allocates memory for a new node, initializes its link to point to itself (self-loop), and returns the pointer.

4.How does the program insert a new term into the polynomial?

→By using the insert_rear() function, which traverses to the last node and links the new node just before the head node.

5.What is the role of the head node in this program?

→The head node acts as a sentinel (dummy) node to simplify insertion and traversal it does not hold any coefficient or power values.

6.How does the program add two polynomials?

→It compares the powers of x and y from both polynomials.

If powers match, it adds their coefficients; if not, it simply copies the term to the result polynomial.

7.What is the use of the evaluate() function?

→It computes the value of the polynomial for given values of x and y using the formula: result += coefficient * pow(x, px) * pow(y, py);

8.What happens if two terms have the same powers in both polynomials?

→Their coefficients are added together to form one term in the resulting polynomial.

9.Why do we use pow(x, px) and pow(y, py)?

→The pow() function (from <math.h>) raises x to the power of px and y to the power of py, which is required to evaluate the polynomial mathematically.

10.What are possible advantages of using dynamic memory allocation in this program?

→allows flexible handling of any number of polynomial terms at runtime without wasting memory — unlike arrays, which have fixed size.

EXPERIMENT 10

Develop a menu driven Program in C for the following operations on **Binary Search Tree (BST)** of Integers

a. Create a BST of **N** Integers: **6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2**
b. Traverse the BST in Inorder, Preorder and Post Order
c. Search the BST for a given element (**KEY**) and report the appropriate message
d. Delete an element (**ELEM**) from BST
e. Exit

ABOUT THE EXPERIMENT:

A **binary search tree** (BST) is a **tree** in which all nodes follows the below mentioned properties

- The left sub-**tree** of a node has key less than or equal to its parent node's key.
- The right sub-**tree** of a node has key greater than or equal to its parent node's key.

Thus, a binary search tree (BST) divides all its sub-trees into two segments; *left* sub-tree and *right* sub-tree and can be defined as

```
left_subtree (keys) ≤ node (key) ≤ right_subtree (keys)
```

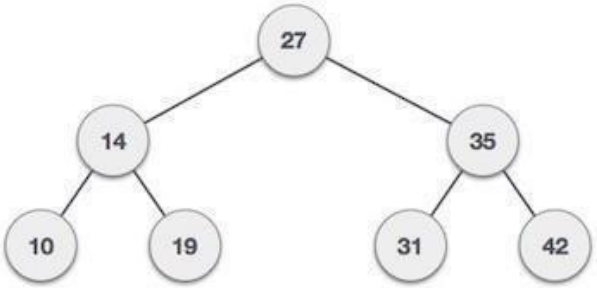


Fig: An example of BST

Following are basic primary operations of a tree which are following.

- **Search** − search an element in a tree.
- **Insert** − insert an element in a tree.
- **Preorder Traversal** − traverse a tree in a preorder manner.
- **Inorder Traversal** − traverse a tree in an inorder manner.
- **Postorder Traversal** − traverse a tree in a postorder manner.

Nodedefinition: Define a node having some data, references to its left and right child nodes.

```
struct node {
   int data;
   struct node *leftChild;
   struct node *rightChild;
};
```

PROGRAM CODE:

```
#include <stdio.h>
#include <stdlib.h>
 struct BST
    {
            int data;
            struct BST *left;
            struct BST *right;
    };

    typedef struct BST NODE;
    NODE *node;
    NODE* createtree(NODE *node, int data)
    {
            if (node == NULL)
            {
                    NODE *temp;
                    temp=
                    (NODE*)malloc(sizeof(NODE));
                    temp->data = data;
                    temp->left = temp->right = NULL;
                    return temp;
            }

            if (data < (node->data))
            {
                    node->left = createtree(node->left, data);
            }
            else if (data > node->data)
            {
                    node -> right = createtree(node->right, data);
            }
            return node;
    }


   NODE* search(NODE *node, int data)
   {
        if(node == NULL)
          printf("\n Element not found");
        else if(data < node->data)
        {
           node->left=search(node->left,  data);
        }
        else if(data > node->data)
       {
           node->right=search(node->right,  data);
        }
       else
            printf("\nElement found is: %d", node->data);
        return node;
   }
```

```
void inorder(NODE *node)

    {
            if(node != NULL)
            {
             inorder(node->left);
             printf("%d\t", node->data);
             inorder(node->right);
            }
    }
 void preorder(NODE *node)
    {
            if(node != NULL)
            {
            printf("%d\t", node->data);
            preorder(node->left);
            preorder(node->right);
            }
    }

    void postorder(NODE *node)
    {
            if(node != NULL)
            {
                    postorder(node->left);
                    postorder(node->right);
                     printf("%d\t", node->data);
             }
    }


void main()
    {
       int data, ch, i, n;
       NODE
       *root=NULL;
       while (1)
         {
                    printf("\n1.Insertion in Binary Search Tree");
                    printf("\n2.Inorder\n3.Preorder\n4.Postorder\n5.Exit");
                    printf("\nEnter your choice: ");
                    scanf("%d", &ch);

                    switch (ch)
                      {
                            case 1: printf("\nEnter N value: " );
                            scanf("%d", &n);
                            printf("\nEnter the values to create BST like(6,9,5,2,8,15,24,14,7,8,5,2)\n");
```

```
                                    for(i=0; i<n; i++)
                                     {
                                            scanf("%d", &data);
                                            root=createtree(root, data);
                                     }
                                break;
                                case 2: printf("\nInorder Traversal: \n");
                                        inorder(root);
                                        break;
                                case 3: printf("\nPreorder Traversal: \n");
                                        preorder(root);
                                        break;
                                case 4: printf("\nPostorder Traversal: \n");
                                        postorder(root);
                                        break;
                                case 5: exit(0);
                                default: printf("\nWrong option");
                                break;
                        }
                }
        }
```

SAMPLE OUTPUT:

1. Insertion in Binary Search Tree
2. Search Element in Binary Search Tree
3. Delete Element in Binary Search Tree
4. Inorder
5. Preorder
6. Postorder
7. Exit

Enter your choice: 1
Enter N value: 12
Enter the values to create BST like(6,9,5,2,8,15,24,14,7,8,5,2)
6 9 5 2 8 15 24 14 7 8 5 2

Insertion in Binary Search Tree
1. Search Element in Binary Search Tree
2. Delete Element in Binary Search Tree
3. Inorder
4. Preorder
5. Postorder
6. Exit

Enter your choice: 4
Inorder Traversal:
2 5 6 7 8 9 14 15 24
1. Insertion in Binary Search Tree
2. Search Element in Binary Search Tree
3. Delete Element in Binary Search Tree
4. Inorder
5. Preorder
6. Postorder
7. Exit

Enter your choice: 5

Preorder Traversal:
6 5 2 9 8 7 15 14 24

1. Insertion in Binary Search Tree
2. Search Element in Binary Search Tree
3. Delete Element in Binary Search Tree
4. Inorder
5. Preorder
6. Postorder
7. Exit

Enter your choice: 6

Postorder Traversal:
2 5 7 8 14 24 15 9 6

1. Insertion in Binary Search Tree
2. Search Element in Binary Search Tree
3. Delete Element in Binary Search Tree Enter your choice:2

Enter the element to search: 24

3. Inorder
4. Preorder
5. Postorder
6. Exit

Element found is: 24

1. Insertion in Binary Search Tree
2. Search Element in Binary Search Tree
3. Delete Element in Binary Search Tree
4. Inorder
5. Preorder
6. Postorder
7. Exit

Enter your choice: 2

Enter the element to search: 50

Element not found

1. Insertion in Binary Search Tree
2. Search Element in Binary Search Tree
3. Delete Element in Binary Search Tree
4. Inorder
5. Preorder
6. Postorder
7. Exit

Enter your choice: 3

Enter the element to search: 15

1. Insertion in Binary Search Tree
2. Search Element in Binary Search Tree
3. Delete Element in Binary Search Tree
4. Inorder
5. Preorder
6. Postorder
7. Exit

Enter your choice: 4

Inorder Traversal:

2 5 6 7 8 9 14 24

1. Insertion in Binary Search Tree
2. Search Element in Binary Search Tree
3. Delete Element in Binary Search Tree
4. Inorder
5. Preorder
6. Postorder
7. Exit

Enter your choice: 5

Preorder Traversal:

6 5 2 9 8 7 24 14

1. Insertion in Binary Search Tree
2. Search Element in Binary Search Tree
3. Delete Element in Binary Search Tree
4. Inorder
5. Preorder
6. Postorder
7. Exit

Enter your choice: 7

VIVA QUESTION:

1. What is a Binary Search Tree (BST)?

→A BST is a binary tree where each node's left child contains values smaller than the parent, and the right child contains values greater than the parent.

2. What are the main operations performed on a BST?

→Insertion, deletion, searching, and traversal (inorder, preorder, postorder).

3. What is the time complexity of searching an element in a BST?
   - Best / Average case: O(log n)
   - Worst case: O(n) (if the tree becomes skewed)
4. Why do we use recursion in BST operations?

→Recursion naturally matches the tree's hierarchical structure, making it easier to navigate left and right subtrees.

5. What is the difference between inorder, preorder, and postorder traversal?
   - Inorder: Left → Root → Right
   - Preorder: Root → Left → Right
   - Postorder: Left → Right → Root
6. What does the inorder traversal of a BST produce?

→It prints the elements in ascending (sorted) order.

7. What happens if you insert duplicate elements into a BST?

→In most implementations (including this program), duplicates are ignored — they are not inserted again.

8. How does the search operation work in a BST? It compares the target value with the current node:

→If smaller → go to left subtree If greater → go to right subtree If equal → element found

9. What is the space complexity of recursive BST operations?

→O(h), where h is the height of the tree (space used by the function call stack).

10. What are the advantages of using a BST?

→Efficient searching, insertion, and deletion (O(log n) average) Maintains sorted order of elements Used in databases, searching algorithms, and symbol tables.

EXPERIMENT 11

Develop a Program in C for the following operations on **Graph(G)** of Cities

a. Create a Graph of **N** cities using Adjacency Matrix.

b. Print all the nodes **reachable** from a given starting node in a digraph using **BFS** method

c. Check whether a given graph is **connected** or not using **DFS** method.

ABOUT THE EXPERIMENT:

Adjacency Matrix

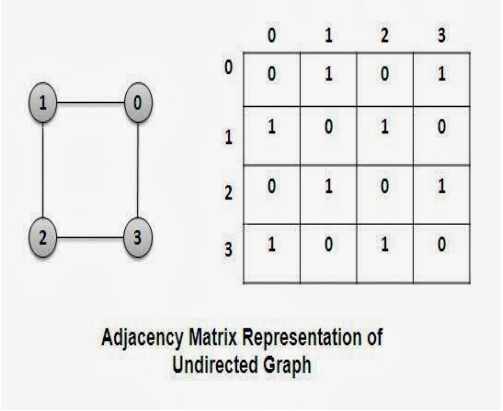
In **graph** theory, computer science, an **adjacency matrix** is a square **matrix** used to **represent** a finite **graph**. The elements of the **matrix** indicate whether pairs of vertices are adjacent or not in the **graph**. In the special case of a finite simple **graph**, the **adjacency matrix** is a (0, 1)-**matrix** with zeros on its diagonal.

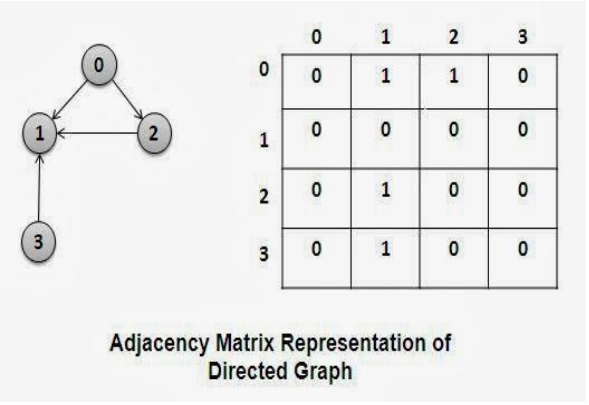
A graph G = (V, E) where v= {0, 1, 2, . . .n-1} can be represented using two dimensional integer array of size n x n.

a[20][20] can be used to store a graph with 20 vertices.
a[i][j] = 1, indicates presence of edge between two vertices i and j. a[i][j] = 0, indicates absence of edge between two vertices i and j.

- A graph is represented using square matrix.
- Adjacency matrix of an undirected graph is always a symmetric matrix, i.e. an edge (i, j) implies the edge (j, i).
- Adjacency matrix of a directed graph is never symmetric, adj[i][j] = 1 indicates a directed edge from vertex i to vertex j.

An example of adjacency matrix representation of an undirected and directed graph is given below:

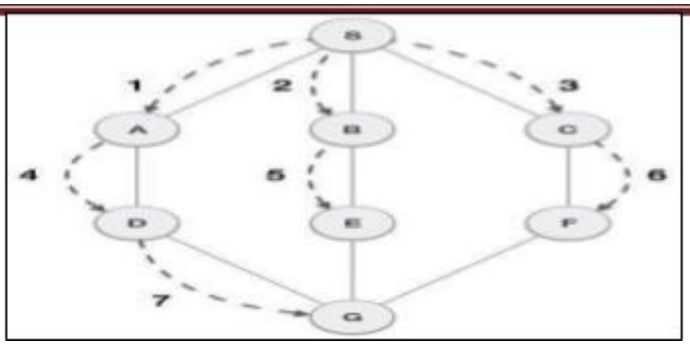




BFS

Breadth-first search (**BFS**) is an algorithm for traversing or searching tree or graph data structures. It starts at the tree root and explores the neighbor nodes first, before moving to the next level neighbors.

Breadth First Search algorithm(BFS) traverses a graph in a breadth wards motion and uses a queue to remember to get the next vertex to start a search when a dead end occurs in any iteration.



As in example given above, BFS algorithm traverses from A to B to E to F first then to C and G lastly to D. It employs following rules.

- **Rule 1** − Visit adjacent unvisited vertex. Mark it visited. Display it. Insert it in a queue.
- **Rule 2** − If no adjacent vertex found, remove the first vertex from queue.
- **Rule 3** − Repeat Rule 1 and Rule 2 until queue is empty.

DFS

Depth-first search (DFS) is an algorithm for traversing or searching tree or graph data structures. One starts at the root (selecting some arbitrary node as the root in the case of a graph) and explores as far as possible along each branch before backtracking.

Depth-first search, or **DFS**, is a way to traverse the graph. Initially it allows visiting vertices of the graph only, but there are hundreds of algorithms for graphs, which are based on DFS. Therefore, understanding the principles of depth-first search is quite important to move ahead into the graph theory. The principle of the algorithm is quite simple: to go forward (in depth) while there is such possibility, otherwise to backtrack.

Algorithm

In DFS, each vertex has three possible colors representing its state:


white: vertex is unvisited;

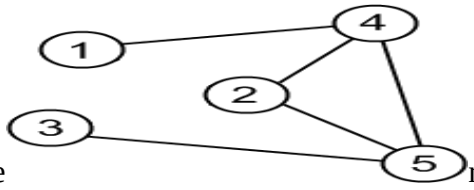

gray: vertex is in progress;


black: DFS has finished processing the vertex.

NB. For most algorithms Boolean classification *unvisited / visited* is quite enough, but we show general case here.

Initially all vertices are white (unvisited). DFS starts in arbitrary vertex and runs as follows:

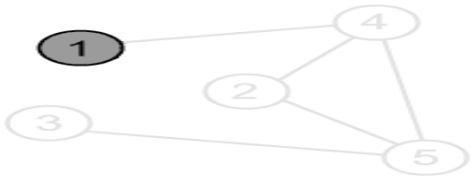
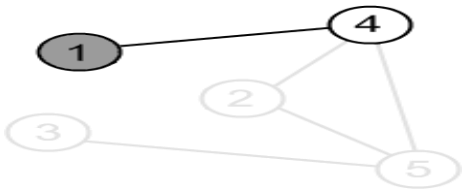
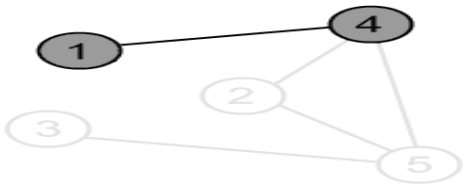
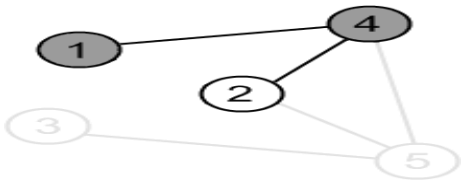
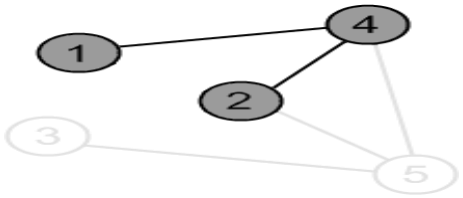
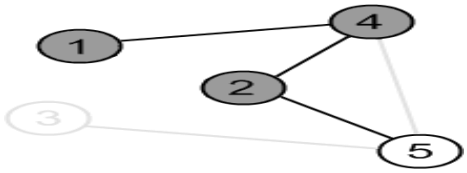
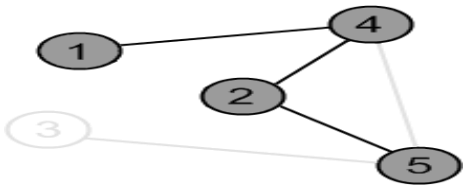
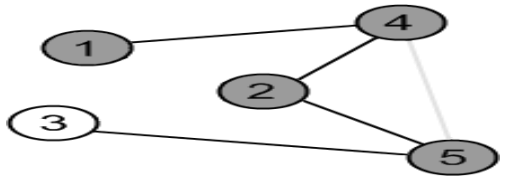
- Mark vertex **u** as gray (visited).
- For each edge **(u, v)**, where **u** is white, run depth-first search for **u** recursively.

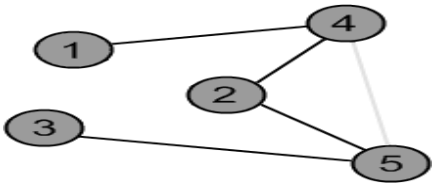
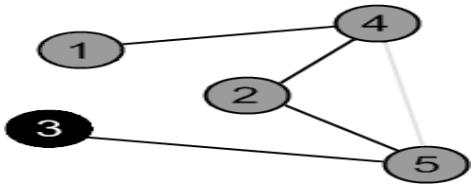
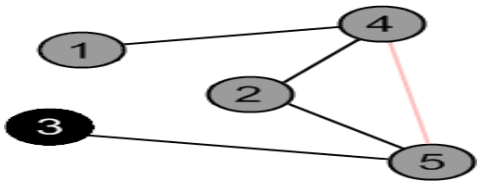
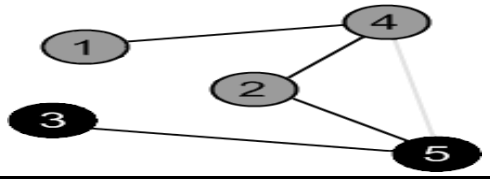
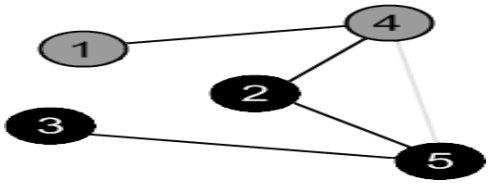
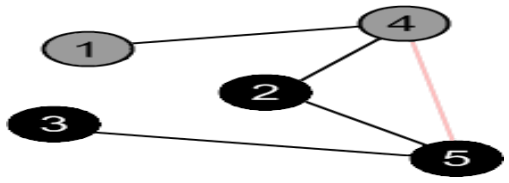
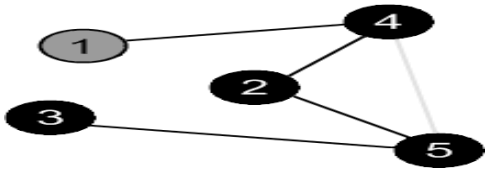
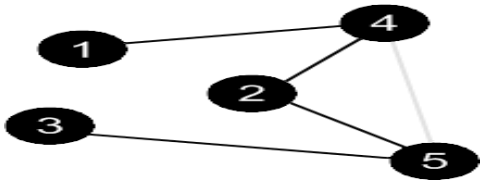


- Mark ve rtex **u** as black and backtrack to the parent.

Example. Traverse a graph shown below, using DFS. Start from a vertex with number 1.

	Source graph.

	Mark a vertex **1** as gray.
	There is an edge **(1, 4)** and a vertex **4** is unvisited. Go there.
	Mark the vertex **4** as gray.
	There is an edge **(4, 2)** and vertex a **2** is unvisited. Go there.
	Mark the vertex **2** as gray.
	There is an edge **(2, 5)** and a vertex **5** is unvisited. Go there.
	Mark the vertex **5** as gray.
	There is an edge **(5, 3)** and a vertex **3** is unvisited. Go there.

	Mark the vertex **3** as gray.
	There are no ways to go from the vertex **3**. Mark it as black and backtrack to the vertex **5**.
	There is an edge **(5, 4)**, but the vertex 4 is gray.
	There are no ways to go from the vertex **5.** Mark it as black and backtrack to the vertex **2**.
	There are no more edges, adjacent to vertex **2.** Mark it as black and backtrack to the vertex **4**.
	There is an edge **(4, 5)**, but the vertex 5 is black.
	There are no more edges, adjacent to the vertex **4.** Mark it as black and backtrack to the vertex **1**.
	There are no more edges, adjacent to the vertex **1.** Mark it as black. DFS is over.

ALGORITHM:

Step 1: Start.
Step 2: Input the value of N nodes of the graph
Step 3: Create a graph of N nodes using adjacency matrix representation.
Step 4: Print the nodes reachable from the starting node using BFS.
Step 5: Check whether graph is connected or not using DFS. Step 6: Stop.

PROGRAM CODE:

```
#include<stdio.h>
int a[10][10], n, m, i, j, source, s[10], b[10];
int visited[10];

void create()
{
        printf("\nEnter the number of vertices of the digraph: ");
        scanf("%d", &n);
        printf("\nEnter the adjacency matrix of the graph:\n");
         for(i=1; i<=n; i++)
        for(j=1; j<=n; j++)
        scanf("%d",&a[i][j);
}

void bfs()
{
   int q[10], u, front=0, rear=-1;
   printf("\nEnter the source vertex to find other nodes reachable or not: ");
   scanf("%d", &source);
   q[++rear] = source;
   visited[source] = 1;
   printf("\nThe reachable vertices are: ");
   while(front<=rear)
        {
            u = q[front++];
            for(i=1; i<=n;i++)
            {
                    if(a[u][i] == 1 && visited[i] == 0)
                    {
                        q[++rear] = i;
                        visited[i] = 1;
                        printf("\n%d",i);
                    }
            }
      }
}

 void dfs(int source)
    {
       int v, top = -1;
       s[++top] = 1;
       b[source] = 1;
```

```
    for(v=1; v<=n; v++)
        {
          if(a[source][v] == 1 && b[v] == 0)
            {
                printf("\n%d -> %d", source, v);
                dfs(v);
            }
        }
}

void main()
{
        int ch;
         while(1)
        {
            printf("\n1.Create Graph\n2.BFS\n3.Check graph connected or not(DFS)\n4.Exit");
            printf("\nEnter your choice: ");
            scanf("%d",&c);
             switch(ch)
            {
                case 1: create();
                        brea;
                case 2: bfs();
                        for(i=1;i<=n;i++)
                        if(visited[i]==0)
                        printf("\the vertex that is not reachable %d" ,i);
                        break;
                case 3: printf("\nEnter the source vertex to find the connectivity: ");
                        scanf("%d", &source);
                        m=1;
                        dfs(source);
                        for(i=1;i<=n;i++)
                        {
                                if(b[i]==0)
                                  m=0;
                        }
                        if(m==1)
                            printf("\n Graph is Connected");
                         else
                            printf("\n Graph is not Connected");
                        break;
                        default: exit(0);
                }
            }
        }
```

SAMPLE OUTPUT:

1. Create Graph

2.BFS

3.Check graph connected or not (DFS)

4.Exit Enter your choice: 1

Enter the number of vertices of the digraph: 4Enter the adjacency matrix of the graph:

0	0	1	1
0	0	0	0
0	1	0	0
0	1	0	0

1. Create Graph

2.BFS

3.Check graph connected or not (DFS)

4.Exit Enter your choice: 2

Enter the source vertex to find other nodes reachable or not: 13 4 2

1. Create Graph

2.BFS

3.Check graph connected or not (DFS)

4.Exit Enter your choice: 3

Enter the source vertex to find the connectivity:

11 -> 3

3 -> 2

1 -> 4

Graph is Connected

1. Create Graph

2.BFS

3.Check graph connected or not (DFS)

4.Exit Enter your choice: 4

VIVA QUESTION:

1. What is the main purpose of the BFS and DFS algorithms in a graph?

   → BFS is used to traverse a graph level by level, while DFS explores as far as possible along each branch before backtracking.

2. Which data structures are used in BFS and DFS respectively?

   → BFS uses a queue (FIFO), and DFS uses a stack (LIFO) or recursion.

3. What is the role of the adjacency matrix in this program?

   → It represents the graph connections, where a[i][j] = 1 means there is an edge from vertex i to vertex j.

4. Why are two separate arrays (visited[] and b[]) used in the program?

   → One (visited[]) is used for BFS and the other (b[]) for DFS to keep their traversals independent.

5. How does the program identify whether the graph is connected or not?

   → It performs a DFS starting from a source vertex.

   If all vertices are visited, the graph is connected; otherwise, it is not connected.

6. What is the time complexity of BFS and DFS?

   → Both have O(V + E) time complexity, where V is the number of vertices and E is the number of edges.

7. What is the difference between a connected and a strongly connected graph?

   → In an undirected graph, "connected" means all vertices are reachable.

   In a directed graph, "strongly connected" means every vertex can reach every other vertex following edge directions.

8. Why do we mark vertices as visited during traversal?

   → To prevent revisiting the same vertex, which could cause infinite loops or redundant processing.

9. What are the advantages and disadvantages of using an adjacency matrix?

   → Advantage: Easy to implement and check edge existence quickly.

   → Disadvantage: Consumes $O(V^2)$ space even for sparse graphs.

10. What will happen if you start BFS from a vertex that cannot reach all others?

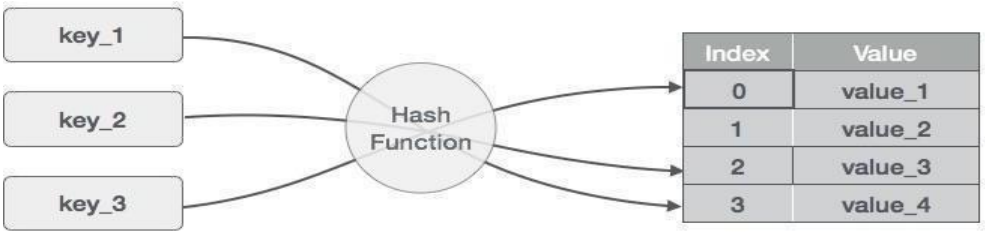
    → Only the reachable vertices will be visited.

    The program will display the remaining vertices as "not reachable."

EXPERIMENT - 12

Given a File of **N** employee records with a set **K** of Keys(4-digit) which uniquely determine the records in file **F**. Assume that file **F** is maintained in memory by a Hash Table(HT) of **m** memory locations with **L** as the set of memory addresses (2-digit) of locations in HT. Let the keys in **K** and addresses in **L** are Integers. Design and develop a Program in C that uses Hash function **H: K ®L** as H(**K**)=**K** mod **m (remainder** method), and implement hashing technique to map a given key **K** to the address space **L.** Resolve the collision (if any) using **linear probing**.

Hashing: Hashing is a technique to convert a range of key values into a range of indexes of an array. We're going to use modulo operator to get a range of key values. Consider an example of hash table of size 20, and following items are to be stored. Item are in (key, value) format.



(1,20) (2,70) (42,80) (4,25) (12,44) (14,32) (17,11) (13,78) (37,98)

S.n.	Key	Hash	Array Index
1	1	1 % 20 = 1	1
2	2	2 % 20 = 2	2
3	42	42 % 20 = 2	2
4	4	4 % 20 = 4	4
5	12	12 % 20 = 12	12
6	14	14 % 20 = 14	14

7	17	17 % 20 = 17	17
8	13	13 % 20 = 13	13
9	37	37 % 20 = 17	17

Linear Probing

As we can see, it may happen that the hashing technique used create already used index of the array. In such case, we can search the next empty location in the array by looking into the next cell until we found an empty cell. This technique is called linear probing.

S.n.	Key	Hash	Array Index	After Linear Probing, Array Index
1	1	1 % 20 = 1	1	1
2	2	2 % 20 = 2	2	2
3	42	42 % 20 = 2	2	3
4	4	4 % 20 = 4	4	4
5	12	12 % 20 = 12	12	12
6	14	14 % 20 = 14	14	14
7	17	17 % 20 = 17	17	17
8	13	13 % 20 = 13	13	13
9	37	37 % 20 = 17	17	18

BasicOperations

Following are basic primary operations of a hash table which are following.

- Search − search an element in a hash table.
- Insert − insert an element in a hash table.
- delete − delete an element from a hash table.

Data Item

Define a data item having some data, and key based on which search is to be conducted in hash table.

ALGORITHM:

Step 1: Start.

Step 2: Given a File of N employee records with a set K of Keys (4-digit) which uniquely determine the records in file F.

Step 3: Assume that file F is maintained in memory by a Hash Table(HT) of m memory locations with L as the set of memory addresses (2-digit) of locations in HT.

Step 3: Let the keys in K and addresses in L are Integers

Step 4: Hash function H: K ®L as H(K)=K mod m (remainder method)

Step 5: Hashing as to map a given key K to the address space L, Resolve the collision (if any) is using linear probing.

Step6: Stop.

PROGRAM CODE:

```
#include <stdio.h>
#include <stdlib.h>
#define M 10 // Size of the hash table

typedef struct
 {
    int key;       // 4-digit employee key
    int address;  // Hash table address (2-digit)
  } EmployeeRecord;

 EmployeeRecord  hashTable[M]; // Hash table of size M

void initializeTable()
  {
      for (int i = 0; i < M; i++)
        {
            hashTable[i].key = -1;  // Mark all slots as empty (-1)
            hashTable[i].address = i; // Assign addresses 0 to M-1
         }
   }
int hashFunction(int key)
{
return key % M;
}
void insert(int key)
 {
    int index = hashFunction(key);
    int originalIndex = index;
    while (hashTable[index].key != -1)
     {
          printf("Collision detected at index %d for key %d. Trying next slot...\n", index, key);
          index = (index + 1) % M;
          if (index == originalIndex)
             {
                   printf("Hash table is full. Cannot insert key %d.\n", key);
```

```
                }
                else
                {
                        return;
                }
            }
            hashTable[index].key = key;
            printf("Key %d inserted at index %d.\n", key, index);
        }
        int search(int key)
        {
            int index = hashFunction(key);
            int originalIndex = index;
            while (hashTable[index].key != -1)
              {
                if (hashTable[index].key == key)
                {
                    return index;
                }
                index (index + 1) % M;
                if (index == originalIndex)
                {
                   break;
                }
            }
        return -1;
      }
    void displayTable()
        {
            printf("\nHash Table:\n");
            for (int i = 0; i < M; i++)
              {
                if (hashTable[i].key != -1)
                  printf("Index %d: Key = %d\n", i, hashTable[i].key);
                else
                  printf("Index %d: Empty\n", i);
             }
        }
        int main()
          {
                initializeTable();
                int choice, key, searchResult;
               while (1)
                {
                      printf("\nMenu:\n");
                      printf("1. Insert employee record\n");
                      printf("2. Search employee record\n");
                      printf("3. Display hash table\n");
                      printf("4. Exit\n");
                      printf("Enter your choice: ");
                      scanf("%d", &choice);
                      switch (choice)
                          {         case 1:
                                          printf("Enter 4-digit employee key to insert: ");
                                          scanf("%d", &key);
                                          insert(key);
```

```
                    break;
                case 2:
                    printf("Enter 4-digit employee key to search: ");
                    scanf("%d", &key);
                    searchResult = search(key);
                    if (searchResult != -1)
                    {
                        printf("Key %d found at index %d.\n", key, searchResult);

                    }
                    else
                    {
                        printf("Key %d not found.\n", key);
                    }
                    break;
                case 3:
                    displayTable();
                    break;
                case 4:  exit(0);
                default:  printf("Invalid choice! Please try again.\n");

            }
        }

        return 0;
    }
```

Sample Output Menu:

Menu:
1. Insert employee record
2. Search employee record
3. Display hash table
4. Exit

Enter your choice: 1
Enter 4-digit employee key to insert: 1234
Key 1234 inserted at index 4.
Menu:
1. Insert employee record
2. Search employee record
3. Display hash table
4. Exit

Enter your choice: 1
Enter 4-digit employee key to insert: 2345 Key 2345 inserted at index 5.
Menu:
1. Insert employee record
2. Search employee record
3. Display hash table
4. Exit

Enter your choice: 1
Enter 4-digit employee key to insert: 3456 Key 3456 inserted at index 6.

Menu:
1. Insert employee record
2. Search employee record
3. Display hash table

4. Exit

Enter your choice: 1

Enter 4-digit employee key to insert: 6534

Collision detected at index 4 for key 6534.

Trying next slot...
Collision detected at index 5 for key 6534.

Trying next slot...

Collision detected at index 6 for key 6534.
Trying next slot...
Key 6534 inserted at index 7.

Menu:
Insert employee record
Search employee record
Display hash table
Exit
Enter your choice: 3

Hash Table:
Index 0: Empty
Index 1: Empty
Index 2: Empty
Index 3: Empty
Index 4: Key = 1234
Index 5: Key = 2345
Index 6: Key = 3456
Index 7: Key = 6534
Index 8: Empty
Index 9: Empty

Menu:
1. Insert employee record
2. Search employee record
3. Display hash table
4. Exit

Enter your choice: 2
Enter 4-digit employee key to search: 1234 Key 1234 found at index 4.

Menu:
1. Insert employee record
2. Search employee record
3. Display hash table
4. Exit

Enter your choice:4

VIVA QUESTION:

1. What is hashing?

→Hashing is a technique used to map large amounts of data (keys) into a smaller range of values (hash table indexes) using a hash function. It allows fast data insertion, deletion, and search.

2. What is a hash function?

→A hash function is a function that takes a key as input and returns an index in the hash table.

In this program**,** **index = key % M;** is the hash function (division method).

3. What is the purpose of M in the program?

M represents the size of the hash table. In this code:

#define M 10

The hash table has 10 slots (from index 0 to 9).

4. What is a collision in hashing?

→A collision occurs when two different keys are mapped to the same index in the hash table by the hash function.

5. What is linear probing?

→Linear probing is a collision resolution technique.

If the desired index is already occupied, the algorithm checks the next slot (index + 1) until it finds an empty one.

6. What happens if the hash table becomes full?

→If all slots are occupied, and a new key needs to be inserted, the program cannot insert it. It displays: Hash table is full. Cannot insert key.

7. How is searching done in linear probing?

→The search function starts at the hash index of the key and checks each consecutive slot (wrapping around if needed) until The key is found, or It returns to the starting point (key not found).

8. What does the statement hashTable[i].key = -1; mean?

→It initializes all hash table slots to -1, which indicates that the slot is empty.

9. What is the time complexity of insertion and search in hashing?

→Best case: O(1) — direct access without collision.

Worst case: O(n) — when many collisions occur and linear probing checks multiple slots.

10. Why is modulo operation (%) used in the hash function?

→The modulo operation ensures that the hash index always lies within the valid range (0 to M-1) of the hash table.

VIVA QUESTIONS AND ANSWERS

Data structures refers to the way data is organized and manipulated. It seeks to find ways to make data access more efficient. When dealing with data structure, we not only focus on one piece of data, but rather different set of data and how they can relate to one another in an organized manner.

Differentiate file structure from storage structure.

Basically, the key difference is the memory area that is being accessed. When dealing with the structure that resides the main memory of the computer system, this is referred to as storage structure. When dealing with an auxiliary structure, we refer to it as file structures.

1. **When is a binary search best applied?**

A binary search is an algorithm that is best applied to search a list when the elements are already in order or sorted. The list is search starting in the middle, such that if that middle value is not the target search key, it will check to see if it will continue the search on the lower half of the list or the higher half. The split and search will then continue in the same manner.

2. **What is a linked list?**

A linked list is a sequence of nodes in which each node is connected to the node following it. This forms a chain-like link of data storage.

3. **How do you reference all the elements in a one-dimension array?**

To do this, an indexed loop is used, such that the counter runs from 0 to the array size minus one. In this manner, we are able to reference all the elements in sequence by using the loop counter as the array subscript.

4. **In what areas do data structures applied?**

Data structures is important in almost every aspect where data is involved. In general, algorithms that involve efficient data structure is applied in the following areas: numerical analysis, operating system, A.I., compiler design, database management, graphics, and statistical analysis, to name a few.

5. **What is LIFO?**

LIFO is short for Last In First Out, and refers to how data is accessed, stored and retrieved. Using this scheme, data that was stored last, should be the one to be extracted first. This also means that in order to gain access to the first data, all the other data that was stored before this first data must first be retrieved and extracted.

6. **What is a queue?**

A queue is a data structures that can simulates a list or stream of data. In this structure, new elements are inserted at one end and existing elements are removed from the other end.

7. What are binary trees?

A binary tree is one type of data structure that has two nodes, a left node and a right node. In programming, binary trees are actually an extension of the linked list structures.

8. Which data structures is applied when dealing with a recursive function?

Recursion, which is basically a function that calls itself based on a terminating condition, makes use of the stack. Using LIFO, a call to a recursive function saves the return address so that it knows how to return to the calling function after the call terminates.

9. What is a stack?

A stack is a data structure in which only the top element can be accessed. As data is stored in the stack, each data is pushed downward, leaving the most recently added data on top.

10. Explain Binary Search Tree

A binary search tree stores data in such a way that they can be retrieved very efficiently. The left subtree contains nodes whose keys are less than the node's key value, while the right subtree contains nodes whose keys are greater than or equal to the node's key value. Moreover, both subtrees are also binary search trees.

11. What are multidimensional arrays?

Multidimensional arrays make use of multiple indexes to store data. It is useful when storing data that cannot be represented using a single dimensional indexing, such as data representation in a board game, tables with data stored in more than one column.

12. Are linked lists considered linear or non-linear data structures?

It actually depends on where you intend to apply linked lists. If you based it on storage, a linked list is considered non-linear. On the other hand, if you based it on access strategies, then a linked list is considered linear.

13. How does dynamic memory allocation help in managing data?

Aside from being able to store simple structured data types, dynamic memory allocation can combine separately allocated structured blocks to form composite structures that expand and contract as needed.

14. What is FIFO?

FIFO is short for First-in, First-out, and is used to represent how data is accessed in a queue. Data has been inserted into the queue list the longest is the one that is removed first.

15. What is an ordered list?

An ordered list is a list in which each node's position in the list is determined by the value of its key component, so that the key values form an increasing sequence, as the list is traversed.

16. What is merge sort?

Merge sort takes a divide-and-conquer approach to sorting data. In a sequence of data, adjacent ones are merged and sorted to create bigger sorted lists. These sorted lists are then merged again to form an even bigger sorted list, which continuous until you have one single sorted list.

17. Differentiate NULL and VOID.

Null is actually a value, whereas Void is a data type identifier. A variable that is given a Null value simply indicates an empty value. Void is used to identify pointers as having no initial size.

18. What is the primary advantage of a linked list?

A linked list is a very ideal data structure because it can be modified easily. This means that modifying a linked list works regardless of how many elements are in the list.

19. What is the difference between a PUSH and a POP?

Pushing and popping applies to the way data is stored and retrieved in a stack. A push denotes data being added to it, meaning data is being "pushed" into the stack. On the other hand, a pop denotes data retrieval, and in particular refers to the topmost data being accessed.

20. What is a linear search?

A linear search refers to the way a target key is being searched in a sequential data structure. Using this method, each element in the list is checked and compared against the target key, and is repeated until found or if the end of the list has been reached.

21. How does variable declaration affect memory allocation?

The amount of memory to be allocated or reserved would depend on the data type of the variable being declared. For example, if a variable is declared to be of integer type, then 32 bits of memory storage will be reserved for that variable.

22. What is the advantage of the heap over a stack?

Basically, the heap is more flexible than the stack. That's because memory space for the heap can be dynamically allocated and de- allocated as needed. However, memory of the heap can at times be slower when compared to that stack.

23. What is a postfix expression?

A postfix expression is an expression in which each operator follows its operands. The advantage of this form is that there is no need to group sub-expressions in parentheses or to consider operator precedence.

24. What is the difference between the HEAP and the STACK?

HEAP is used to store dynamically allocated memory (malloc). STACK stores static data (int, const).

25. Where in memory are HEAP and the STACK located relative to the executing program?

The STACK and HEAP are stored "below" the executing program. The HEAP "grows" toward the program executable while the STACK grows away from it.

26. Describe the data structures of a double-linked list.

A double-linked list structure contains one pointer to the previous record in the list and a pointer to the next record in the list plus the record data.

27. How do you insert a record between two nodes in double-linked list?

Previous R; Data R; Next R; To insert a record (B) between two others (A and C): Previous.B = A; Next.B = C; Next.A =B; Previous.C = B;

28. In which data structure, elements can be added or removed at either end, but not in the middle?

Queue

29. Which one is faster? A binary search of an orderd set of elements in an array or a sequential search of the elements.

Binary search

30. What is a balanced tree?

A binary tree is balanced if the depth of two subtrees of every node never differ by more than one

31. Which data structure is needed to convert infix notations to post fix notations?

stack

32. What is data structure or how would you define data structure?

In programming the term data structure refers to a scheme for organizing related piece of information. Data Structure = Organized Data + Allowed Operations.

33. Which data structures we can implement using link list?

queue and stack

34. List different types of data structures?

Link list, queue, stack, trees, files, graphs